252-0027

Introduction to Programming

Exercises

Week 9: Classes, Linked Objects

First name Last name
Department of Computer Science
ETH Zurich

Organization

- My name: *First name Last name*
- For questions: ETH email
- Please put [EProg26] in the email subject
- New exercises: Tuesday evening (normally)
- Submission: Tuesday of the following week, 23:59
  - Always submit via Git
  - Solutions in a separate Git project

Solving Problems

Solving Problems: Labyrinth

Rooms and solution paths

A labyrinth consists of rooms represented by the class `Room`. The attribute `int age` gives the non-negative age of a room. The array `Room[] doorsTo` contains the rooms directly reachable through a door. Doors work in one direction only. A room is an *exit* if `doorsTo.length == 0`. Neither the array nor its entries are ever `null`.

For both subtasks, the starting room `room` is not `null`. The labyrinth is finite and has no directed cycles. Different paths may, however, merge into the same room. A *solution path* $r_1, \ldots, r_N$ starts at `room`, ends at an exit, and follows a door at every step: for $1 \leq i < N$, r_{i+1} occurs in r_i`.doorsTo`. If the starting room is already an exit, the path consists of that room alone.

Solving Problems: Labyrinth

Exit if:

- `r.doorsTo.length == 0`

Solution path $r_1, \ldots, r_N$ if:

- r_1 is `room` (the room we are given)
- r_N is an exit
- r_i and r_{i+1} are connected by a door.

For both subtasks, the starting room `room` is not `null`. The labyrinth is finite and has no directed cycles. Different paths may, however, merge into the same room. A *solution path* $r_1, \ldots, r_N$ starts at `room`, ends at an exit, and follows a door at every step: for $1 \leq i < N$, r_{i+1} occurs in `r[i].doorsTo`. If the starting room is already an exit, the path consists of that room alone.

Solving Problems: Labyrinth

a) Prefix sums

The method `public static boolean task1(Room room)` should return `true` exactly when there is a solution path $r_1, \ldots, r_N$ for which **every prefix sum** of the ages is not divisible by 3:

$$\sum_{j=1}^{i} r_j.\texttt{age} \not\equiv 0 \pmod 3 \qquad \text{for all } 1 \leq i \leq N.$$

This includes the age of the starting room and the sum of the entire path. A path with ages $[1, 1, 2]$ satisfies the condition (sums $1, 2, 4$), whereas a path with $[1, 2, 1]$ does not (the second sum is 3).

b) Same age sequence, different rooms

The method `public static boolean task2(Room room)` should return `true` exactly when there are two solution paths $r_1, \ldots, r_N$ and $s_1, \ldots, s_N$ such that:

- both paths have the same length and $r_i.\texttt{age} = s_i.\texttt{age}$ for all $1 \leq i \leq N$;
- at least one position i has different room references, that is, `r`$_i$ `!=` `s`$_i$.

Different rooms may have the same attribute values. Two doors leading to the same room do not, by themselves, yield different room paths. The condition from a) does **not** apply to b).

Important!

Template and tests: Open this Gradle project folder in IntelliJ. Complete the methods in `Labyrinth.java`. You may add helper methods as well as methods and fields to `Room`. Run `main` for an example comparison and `LabyrinthTest` for the JUnit tests. The unfinished template initially returns `false`, so not all tests will pass yet.

```java
public class Room {

    private int age;          // Age or the room, >= 0
    public Room[] doorsTo;    // Doors to other rooms, never null

    // PRE: age >= 0; doorsTo and all its elements are non-null.
    public Room(int age, Room... doorsTo) {
        this.age = age;
        this.doorsTo = doorsTo;
    }
    public boolean isExit() { // Exit if there are no doors
         return doorsTo.length == 0;
    }

    public int getAge() { // Return the room's age
        return age;
```

What is in the `Labyrinth` class?

```java
public class Labyrinth {

    public static boolean task1(Room room) {
        // TODO
        return false;
    }

    public static boolean task2(Room room) {
        // TODO
        return false;
    }
}
```

Solving Problems: Labyrinth

The method `public static boolean task1(Room room)` should return `true` exactly when there is a solution path $r_1, \ldots, r_N$ for which **every prefix sum** of the ages is not divisible by 3:

$$\sum_{j=1}^{i} r_j.\text{age} \not\equiv 0 \pmod 3 \qquad \text{for all } 1 \le i \le N.$$

This includes the age of the starting room and the sum of the entire path. A path with ages $[1, 1, 2]$ satisfies the condition (sums $1, 2, 4$), whereas a path with $[1, 2, 1]$ does not (the second sum is 3).

How do we solve the problem?

- **Start in a room of a directed, acyclic labyrinth. Find paths to an exit. Task 1 checks prefix sums. Task 2 seeks distinct paths with matching ages.**



Identify subproblems:

- Assume we are in a room r_i how do we know whether the path $r_1, \ldots, r_i$ is a solution?

- TS property: every prefix starting at the initial room has an age sum not divisible by 3. This does not refer to arbitrary segments within the path.



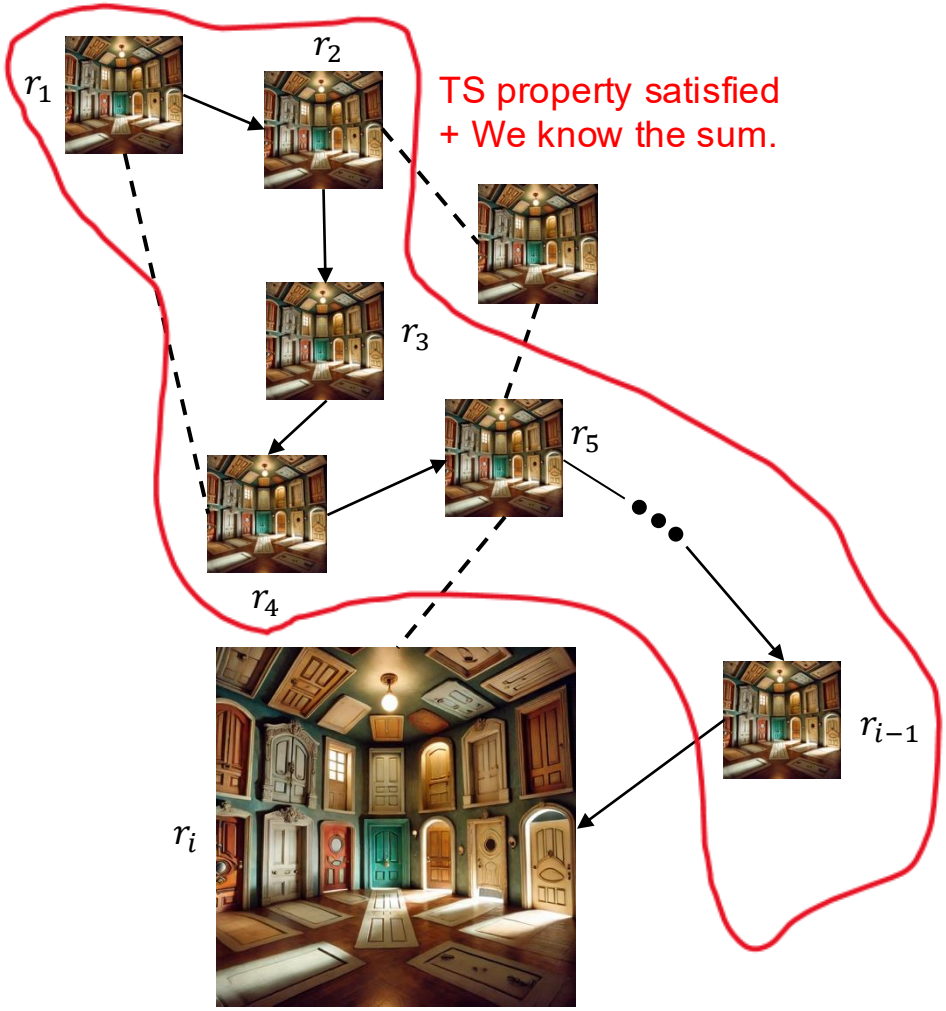
Identify subproblems (Attempt 1):

Current path: TS property satisfied.

Is that enough to append a room?

No: current sum 22, next age 11.
22 % 3 != 0
(22 + 11) % 3 == 0

We also need to know the sum.



Identify subproblems (Attempt 2):

The current path satisfies the TS property.

sum stores the age sum of this path.

For the next room, check:
(sum + age) % 3 != 0

Only then do we continue the search.

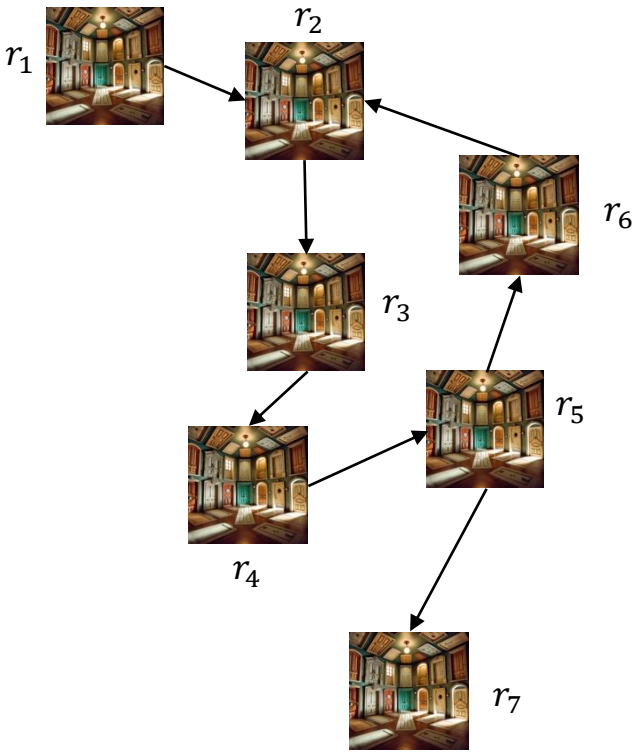
```java
// PRE: sum = sum of previous rooms; all satisfy the TS property
public static boolean solve1(Room room, int sum) {
    sum = sum + room.getAge();

    if (sum % 3 == 0) { // TS property violated: stop exploring this path
        return false;
    }
    if (room.isExit()) { // Exit reached: done
        return true;
    }
    for (int i = 0; i < room.doorsTo.length; i++) { // Check all neighboring rooms (no cycles)
        if (solve1(room.doorsTo[i], sum)) { // Path through this door leads to an exit
            return true;
        }
    }
    return false; // No valid path to an exit found
}
```

How do we use this method?

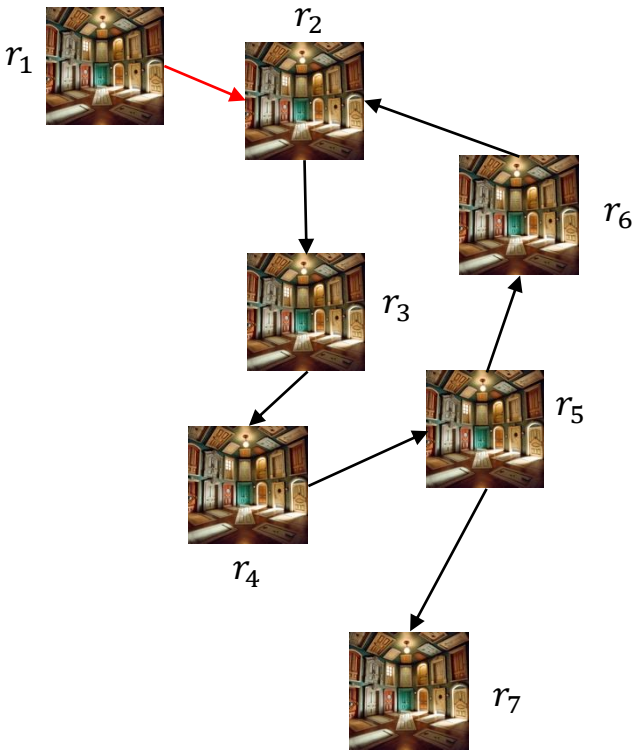
```java
public static boolean solve1(Room room, int sum) {
    // ...
}

public static boolean task1(Room room) {
    return solve1(room, 0); // Start search at room with initial sum 0
}
```



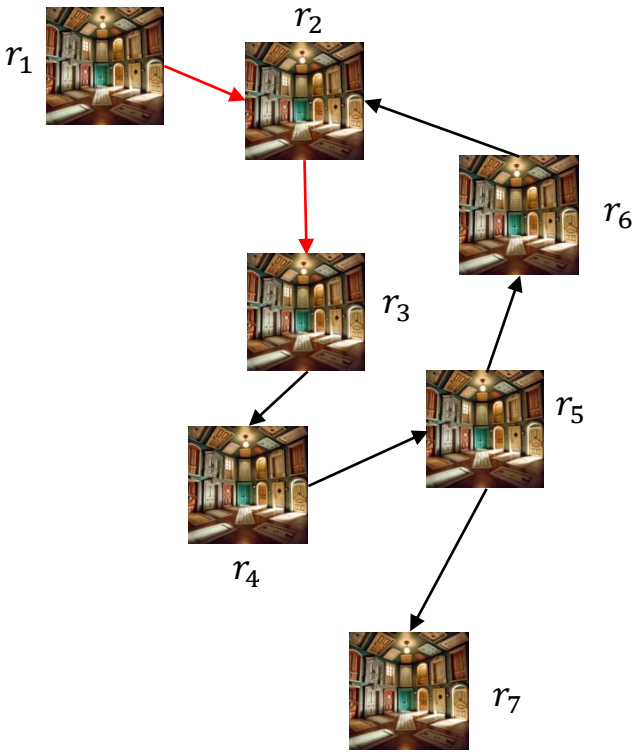
Why no cycles?

- Cycles can cause infinite recursion unless we track the rooms already on our path.



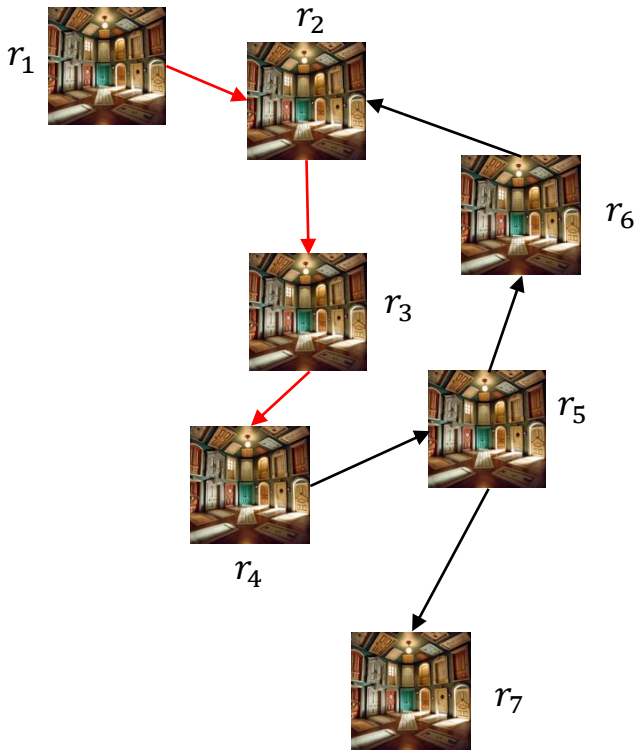
Why no cycles?

- Cycles can cause infinite recursion unless we track the rooms already on our path.



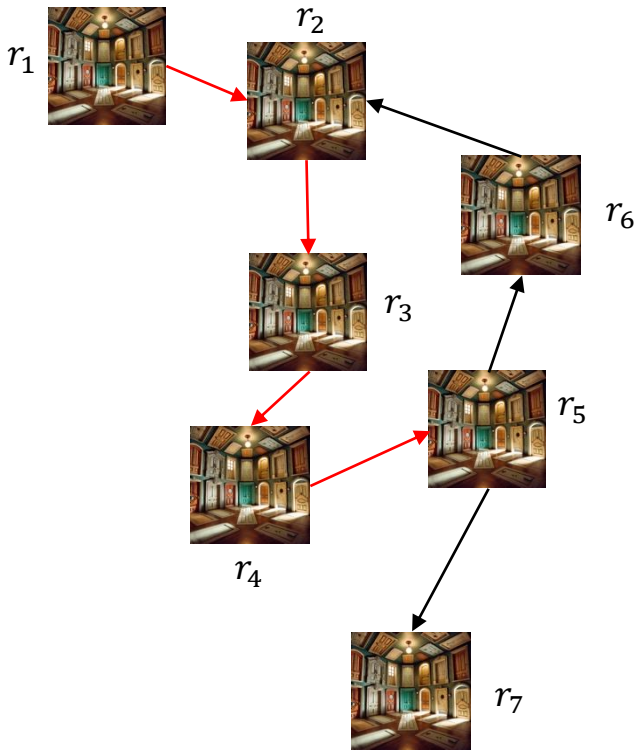
Why no cycles?

- Cycles can cause infinite recursion unless we track the rooms already on our path.



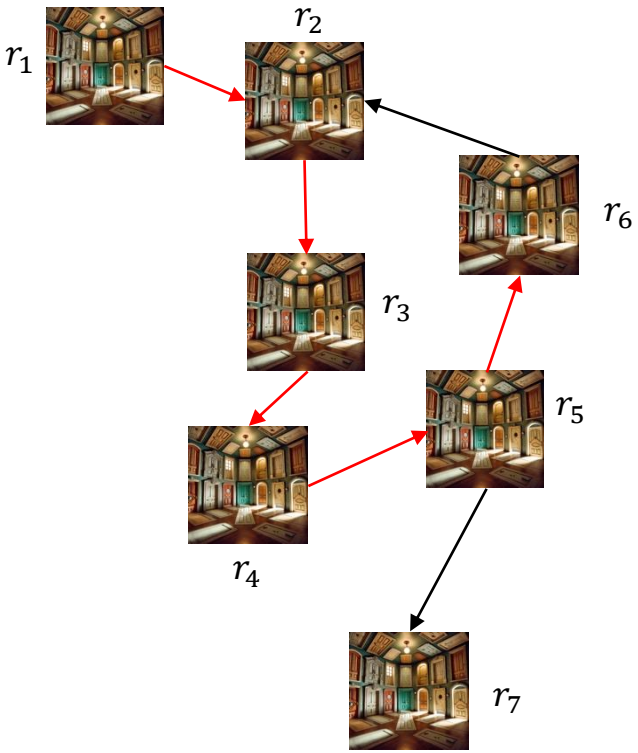
Why no cycles?

- Cycles can cause infinite recursion unless we track the rooms already on our path.



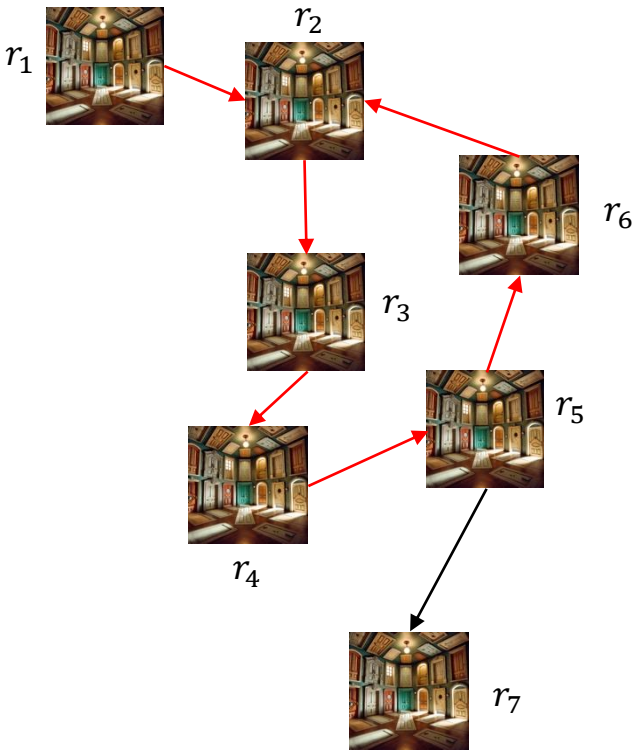
Why no cycles?

- Cycles can cause infinite recursion unless we track the rooms already on our path.



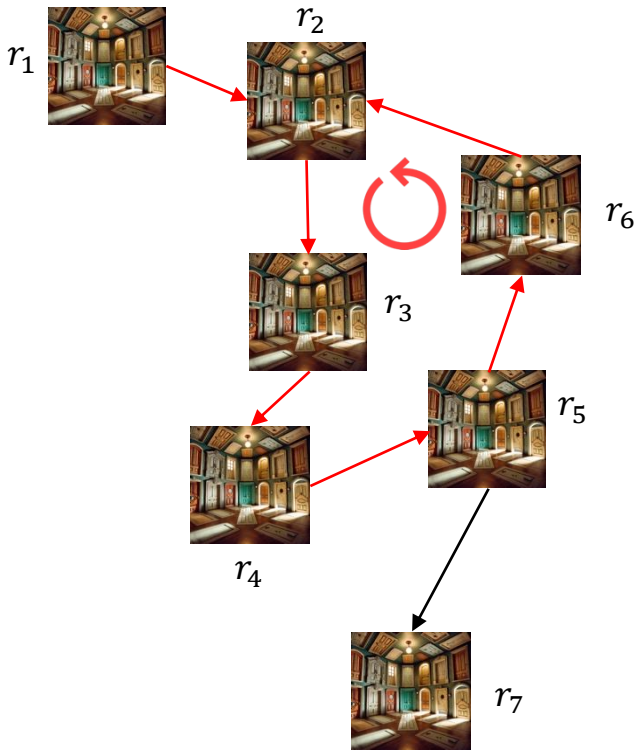
Why no cycles?

- Cycles can cause infinite recursion unless we track the rooms already on our path.



Why no cycles?

- Cycles can cause infinite recursion unless we track the rooms already on our path.



Why no cycles?

- Cycles can cause infinite recursion unless we track the rooms already on our path.

Solving Problems: Labyrinth

The method `public static boolean task2(Room room)` should return `true` exactly when there are two solution paths $r_1, \ldots, r_N$ and $s_1, \ldots, s_N$ such that:

- both paths have the same length and r_i.`age` $= s_i$.`age` for all $1 \leq i \leq N$;
- at least one position i has different room references, that is, `r`$_i$ `!=` `s`$_i$.

Different rooms may have the same attribute values. Two doors leading to the same room do not, by themselves, yield different room paths. The condition from a) does **not** apply to b).

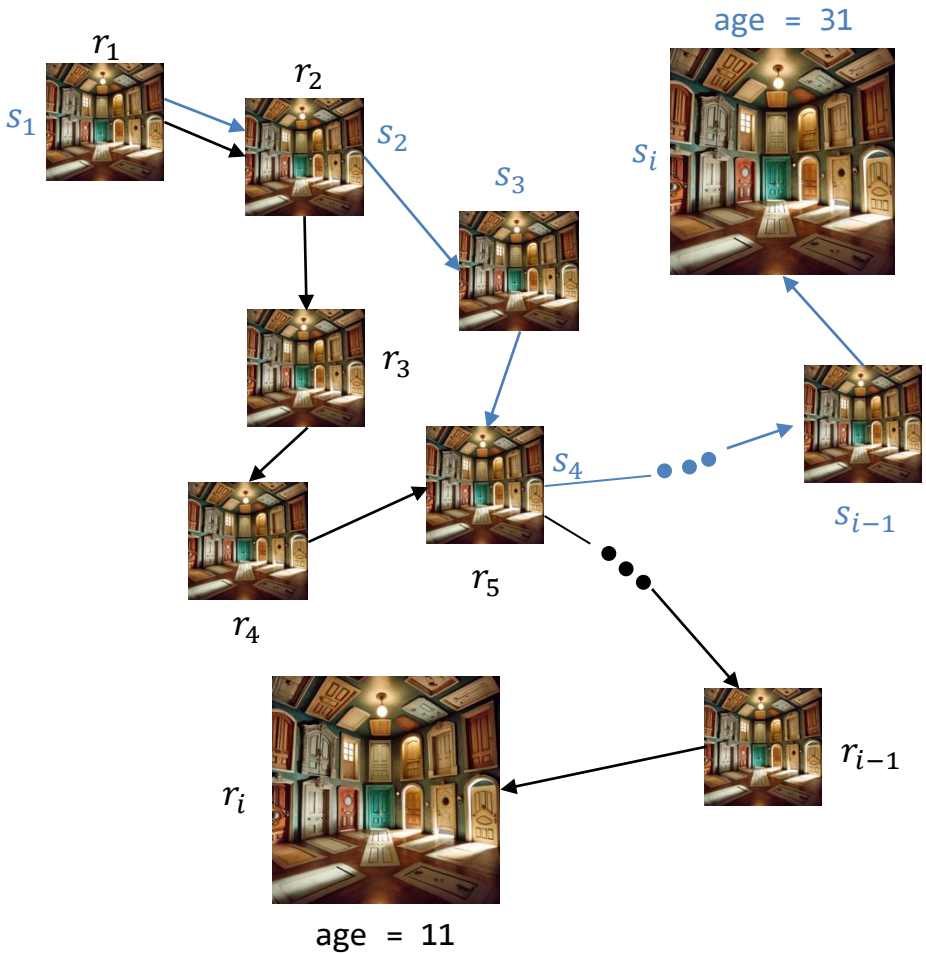
How do we solve the problem?

- **Recursive solution: identify the subproblems.**



Identify subproblems:

- We now consider two rooms r_i and s_i and two paths $r_1, \ldots, r_i$ and $s_1, \ldots, s_i$ where r_1 and s_1 are both the starting room room.
- What must we know about the prefixes $r_1, \ldots, r_{i-1}$ and $s_1, \ldots, s_{i-1}$?
- Age condition: at each position i, r_i and s_i have the same age: r_i.getAge() == s_i.getAge().



Identify subproblems:

- If the paths have already diverged, check whether r_i and s_i are both exits with the same age. If so, we are done.
- Otherwise, explore all possible path pairs (r_{i+1}, s_{i+1}).
- How do we track whether the paths have diverged?

With a boolean parameter

```java
// PRE: Both paths have equal length and age sequence; samePath = identical path so far
public static boolean solve2(Room room1, Room room2, boolean samePath) {

    if (room1.isExit() && room2.isExit() && !samePath) { // Two distinct paths reach an exit
        return true;
    }
    for (int i = 0; i < room1.doorsTo.length; i++) { // Check all pairs of rooms
        for (int k = 0; k < room2.doorsTo.length; k++) {
            if (room1.doorsTo[i].getAge() == room2.doorsTo[k].getAge()) { // Same age?
                if (solve2(room1.doorsTo[i], room2.doorsTo[k],
                        samePath && room1.doorsTo[i] == room2.doorsTo[k])) { // Same path only if rooms are identical
                    return true; // Valid pair of paths found
                }
            }
        }
    }
    return false; // No valid pair of paths found
}
```

How do we use these methods?

```java
public static boolean solve1(Room room, int sum) {
    // ...
}

public static boolean task1(Room room) {
    return solve1(room, 0); // Start with sum 0
}

public static boolean solve2(Room room1, Room room2, boolean samePath) {
    // ...
}

public static boolean task2(Room room) {
    return solve2(room, room, true); // Both paths start identically in the same room
}
```

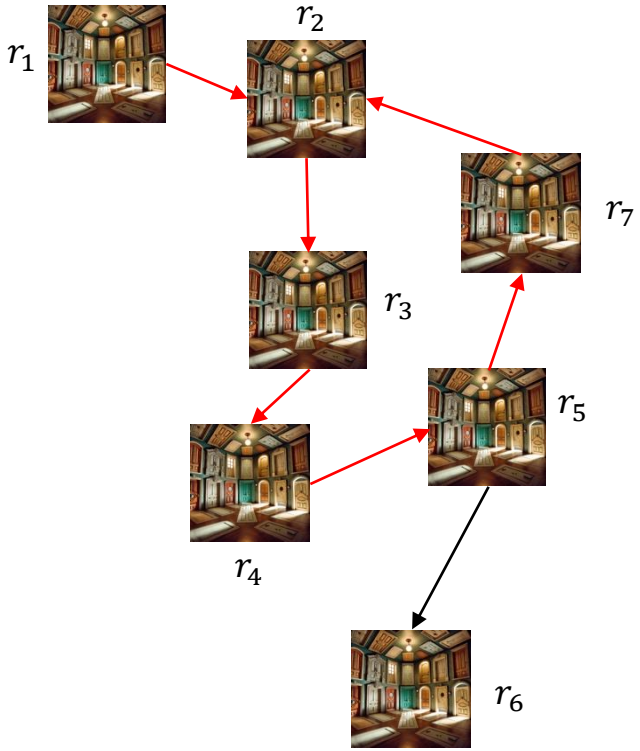
Working with directed cycles

Solving Problems: Labyrinth (modified)

Rooms and solution paths

A labyrinth consists of rooms represented by the class `Room`. The attribute `int age` gives the non-negative age of a room. The array `Room[] doorsTo` contains the rooms directly reachable through a door. Doors work in one direction only. A room is an *exit* if `doorsTo.length == 0`. Neither the array nor its entries are ever `null`.

For both subtasks, the starting room `room` is not `null`. The labyrinth is finite and ~~has no directed cycles.~~ Different paths may, however, merge into the same room. A *solution path* $r_1, \ldots, r_N$ starts at `room`, ends at an exit, and follows a door at every step: for $1 \leq i < N$, r_{i+1} occurs in r_i`.doorsTo`. If the starting room is already an exit, the path consists of that room alone.



Directed cycles cause problems

- Without cycle detection, recursion may continue forever. Should visited mean ever visited or on the current path?
- This extension searches for simple paths: no room occurs twice.
- **Later: sets**
- **Now: a visited attribute.**

Solving Problems: Labyrinth

a) Prefix sums

The method `public static boolean task1(Room room)` should return `true` exactly when there is a solution path $r_1, \ldots, r_N$ for which **every prefix sum** of the ages is not divisible by 3:

$$\sum_{j=1}^{i} r_j.\texttt{age} \not\equiv 0 \pmod 3 \qquad \text{for all } 1 \le i \le N.$$

This includes the age of the starting room and the sum of the entire path. A path with ages $[1, 1, 2]$ satisfies the condition (sums $1, 2, 4$), whereas a path with $[1, 2, 1]$ does not (the second sum is 3).

b) Same age sequence, different rooms

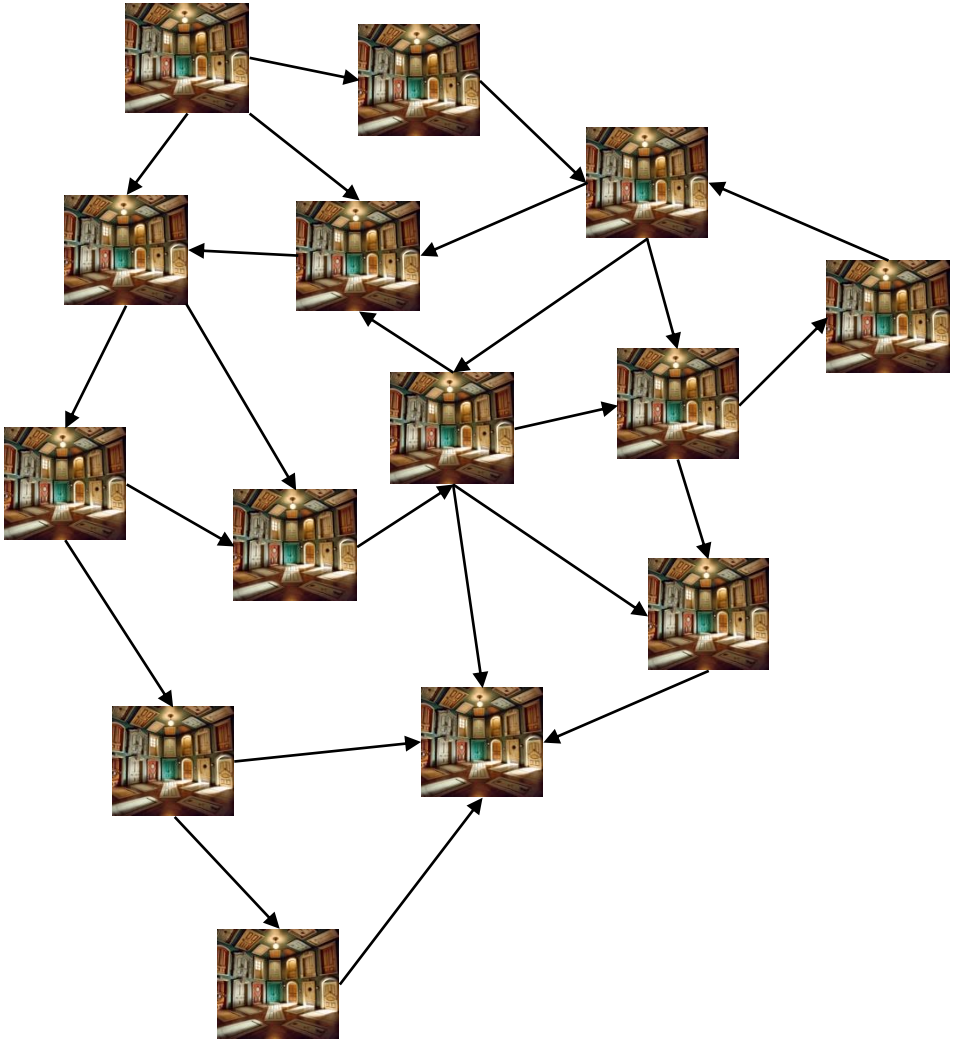
The method `public static boolean task2(Room room)` should return `true` exactly when there are two solution paths $r_1, \ldots, r_N$ and $s_1, \ldots, s_N$ such that:

- both paths have the same length and $r_i.\texttt{age} = s_i.\texttt{age}$ for all $1 \le i \le N$;
- at least one position i has different room references, that is, `r`$_i$ `!=` `s`$_i$.

Different rooms may have the same attribute values. Two doors leading to the same room do not, by themselves, yield different room paths. The condition from a) does **not** apply to b).

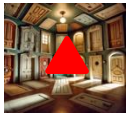
Template and tests: Open this Gradle project folder in IntelliJ. Complete the methods in `Labyrinth.java`. You may add helper methods as well as methods and fields to `Room`. Run `main` for an example comparison and `LabyrinthTest` for the JUnit tests. The unfinished template initially returns `false`, so not all tests will pass yet.

We can use this!

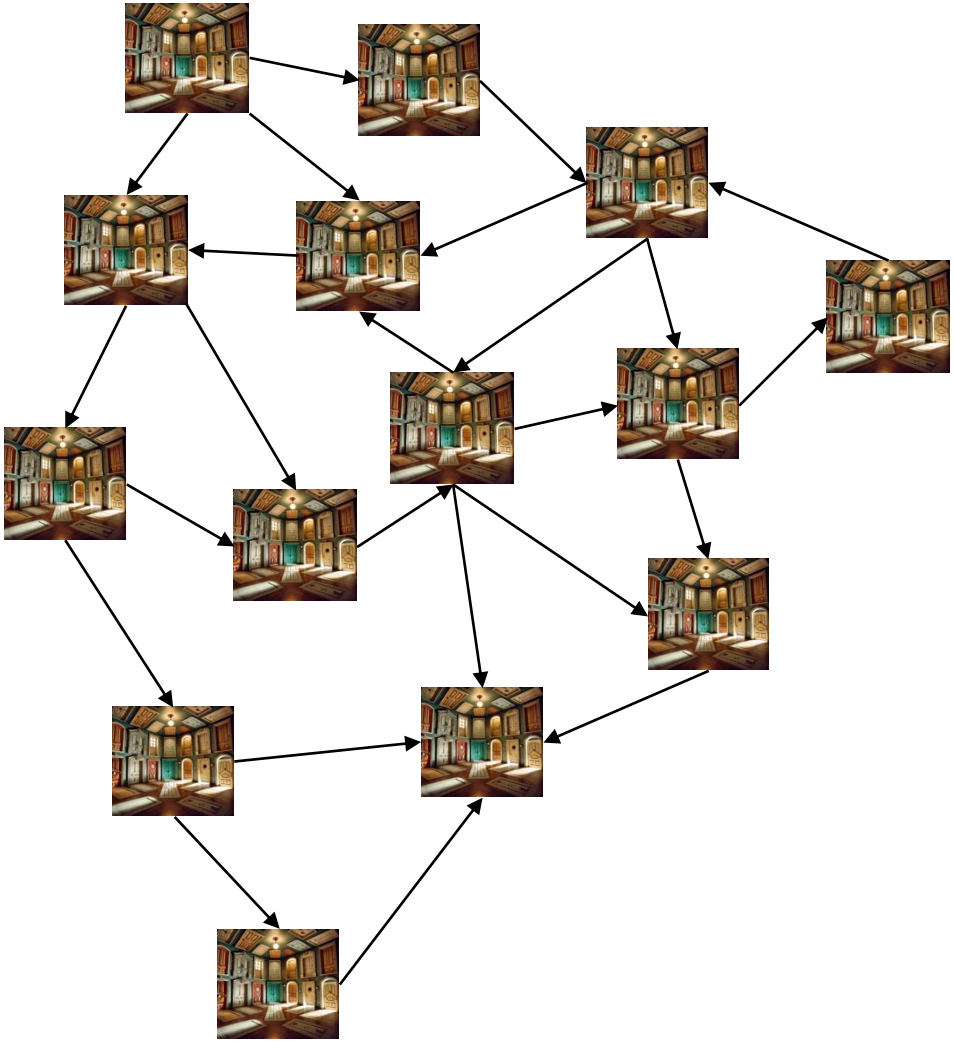


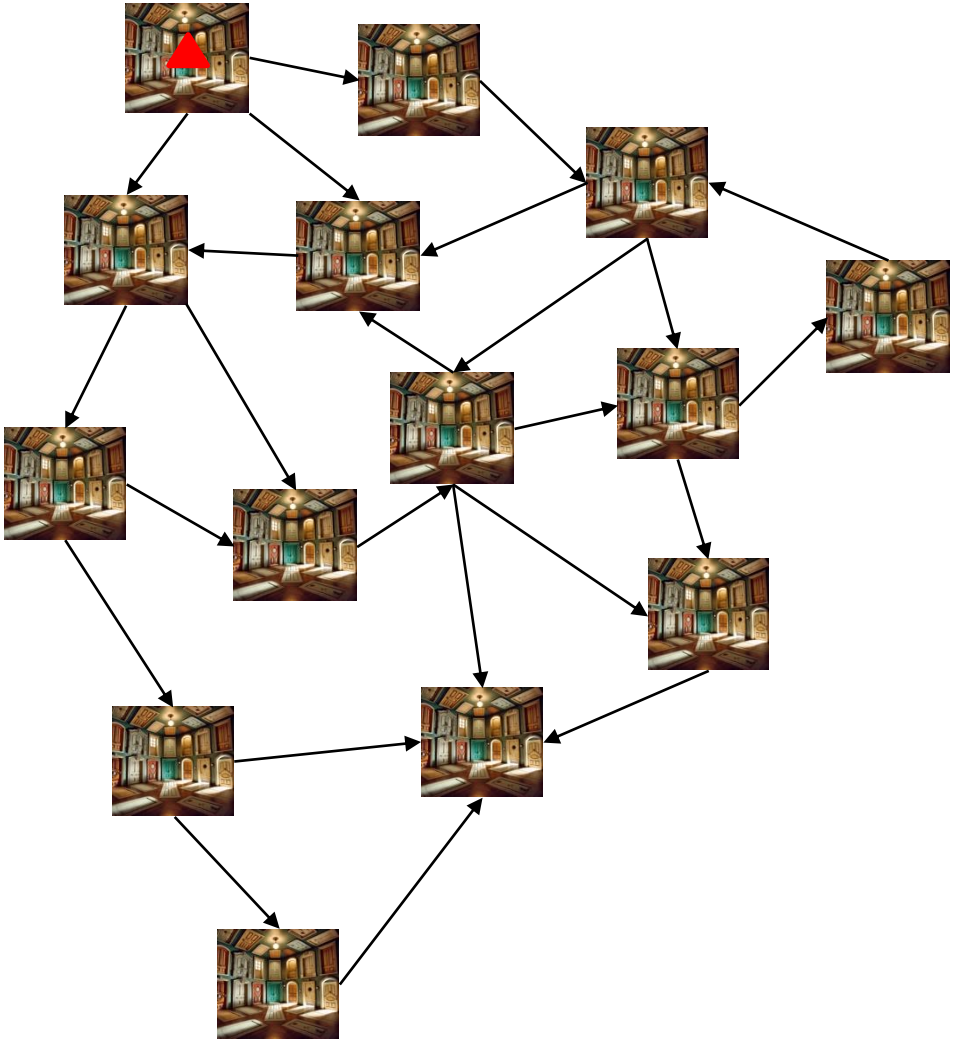


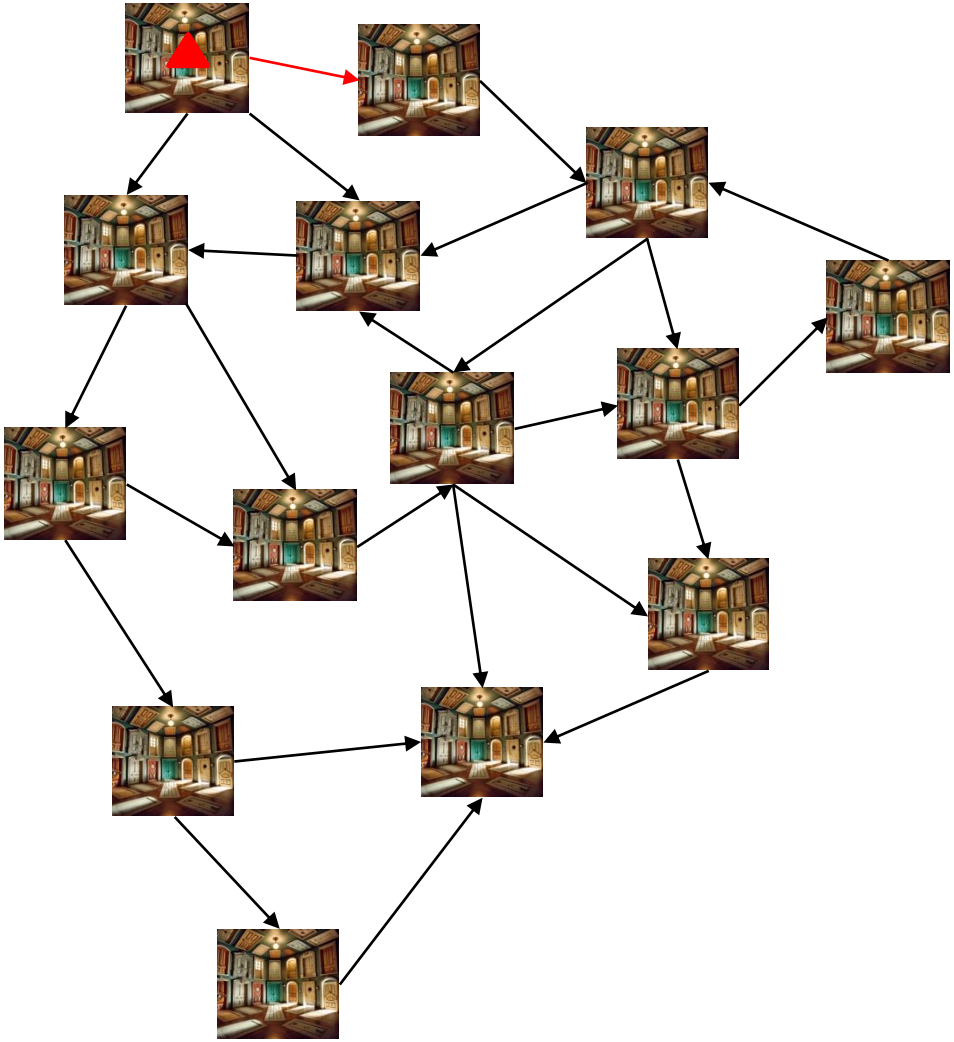
not visited

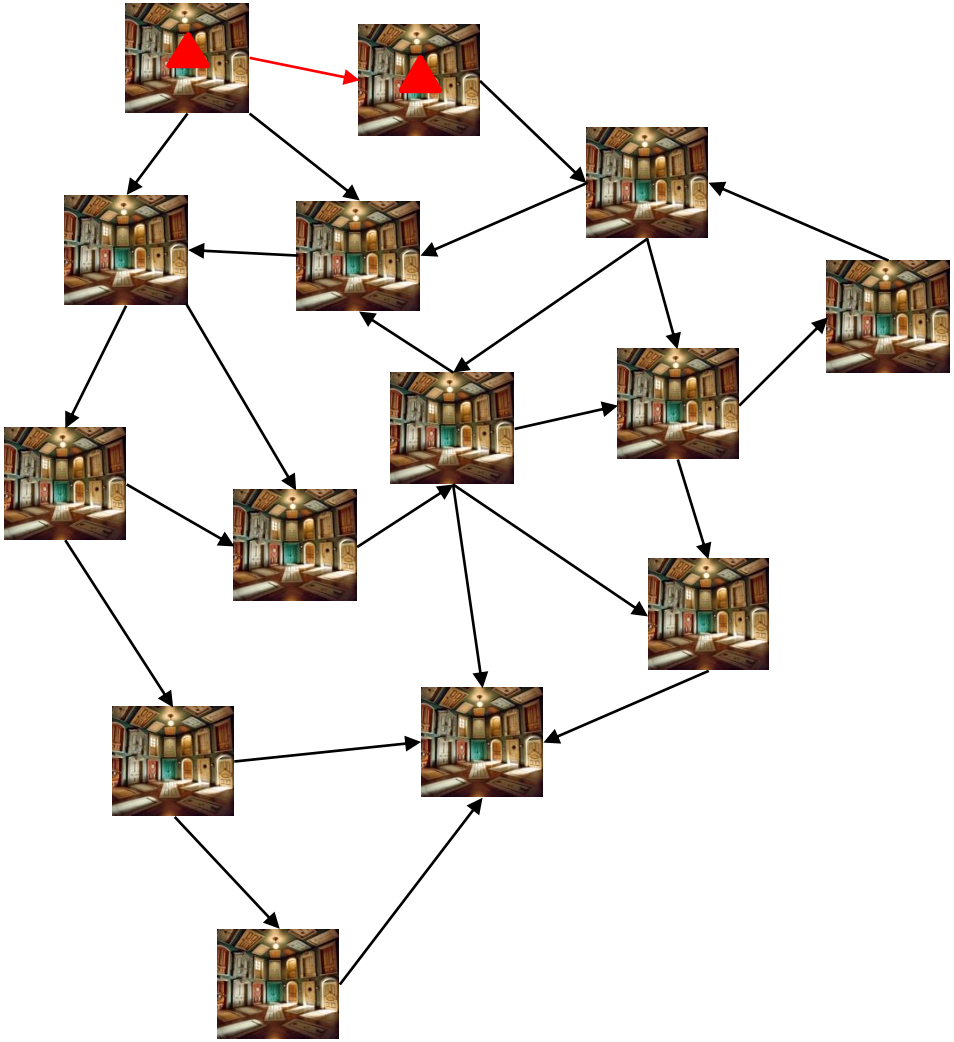


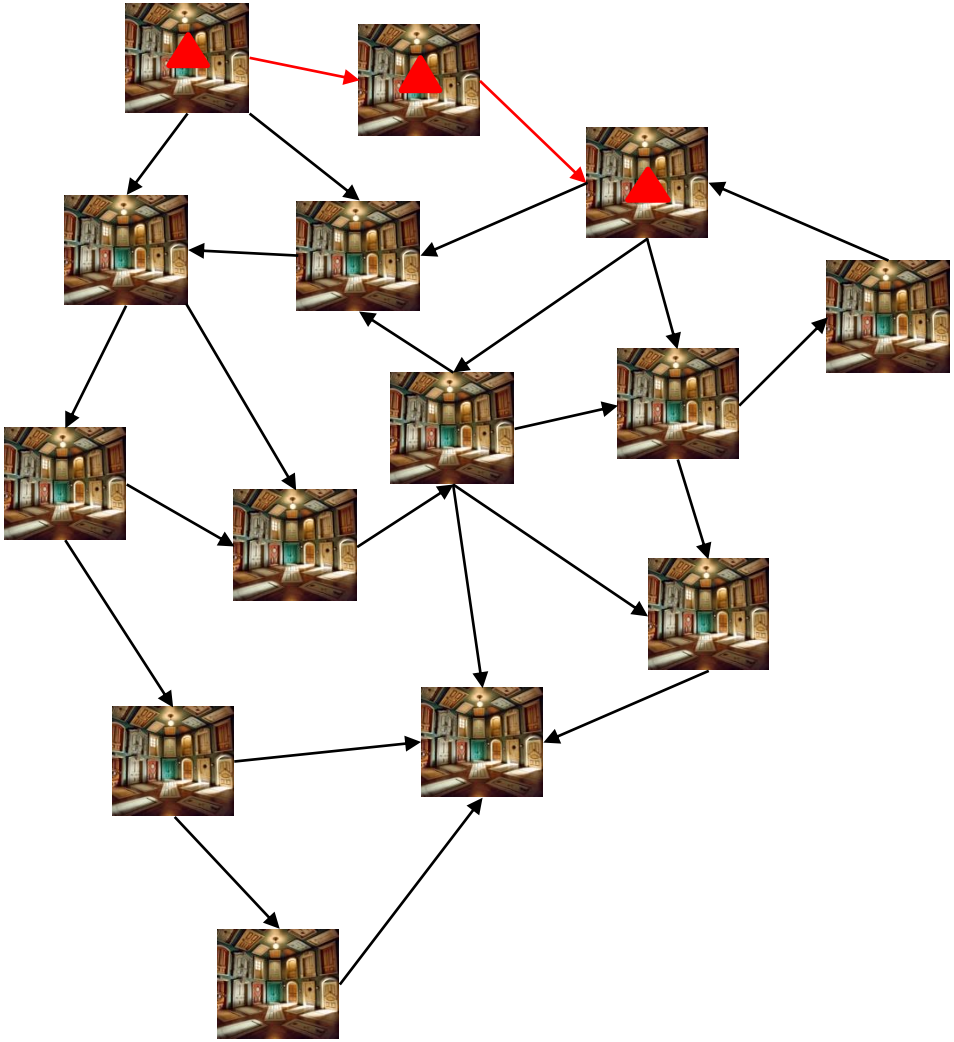
visited

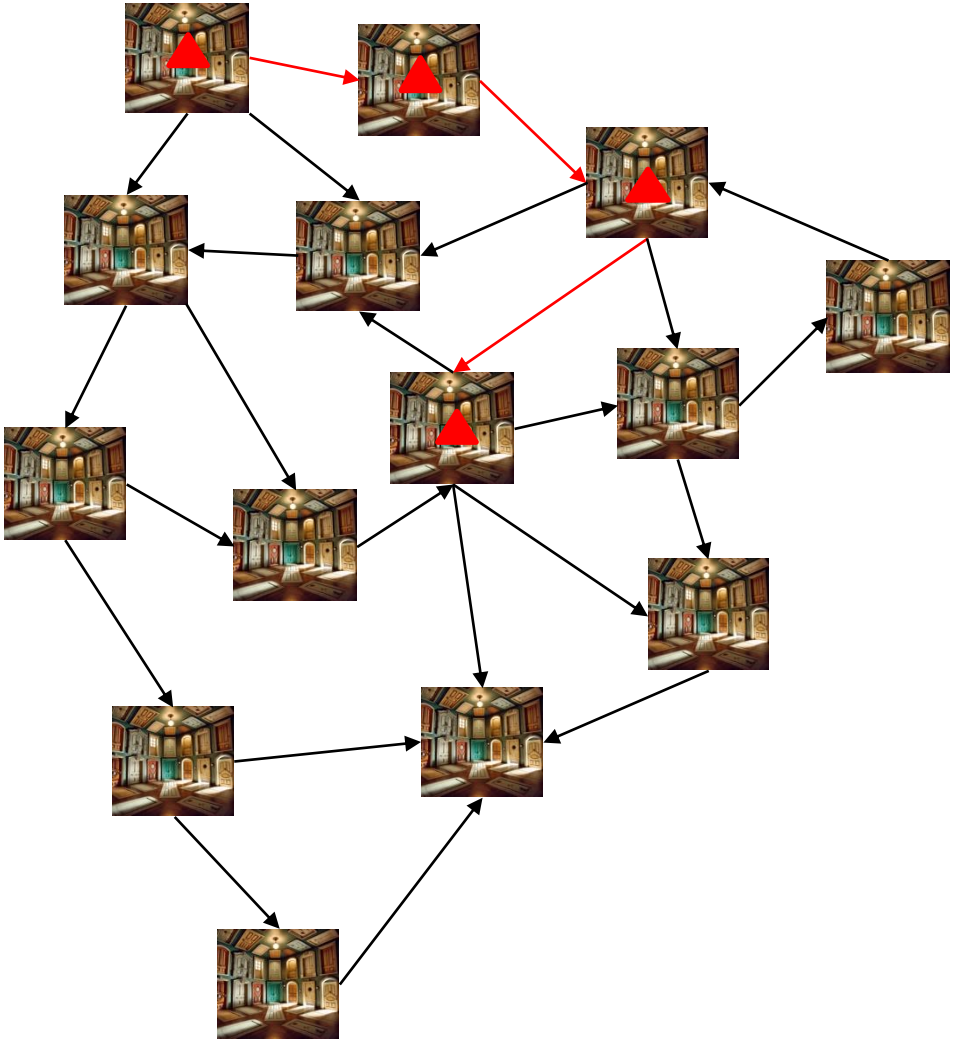


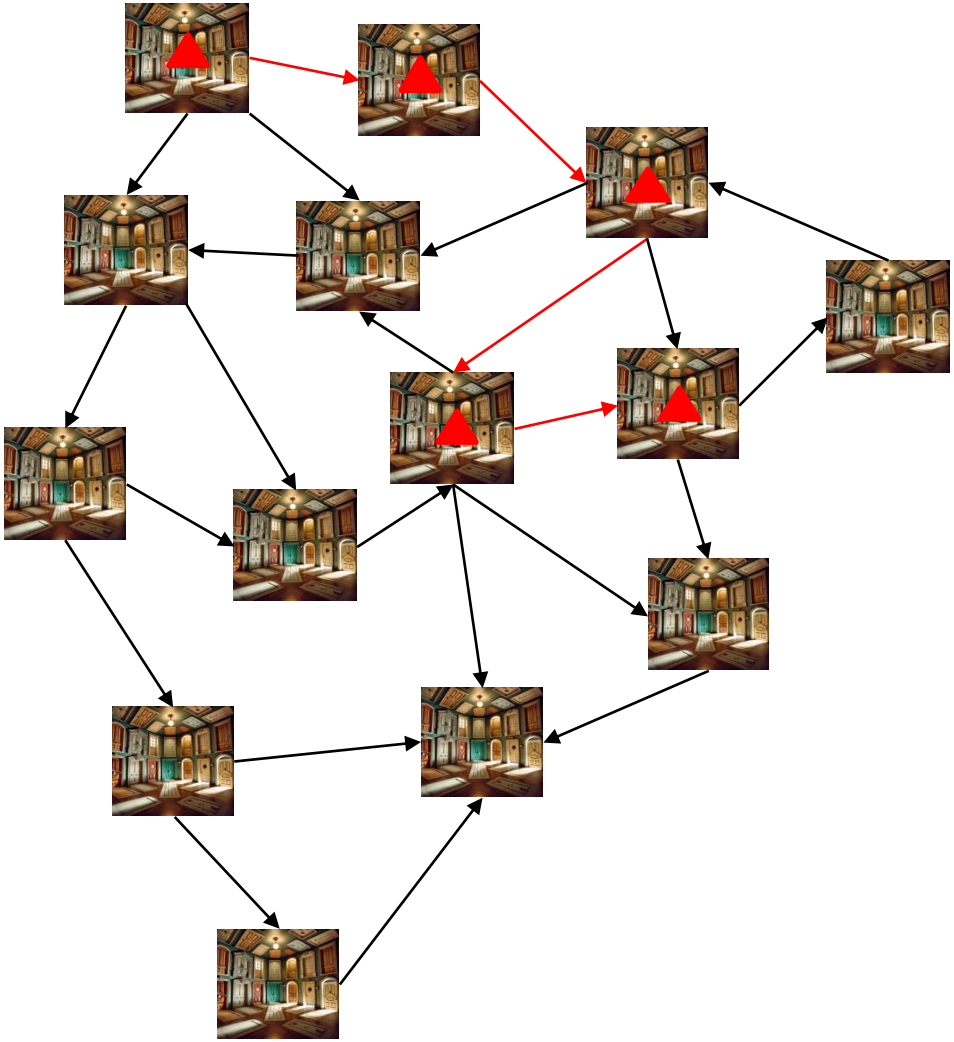


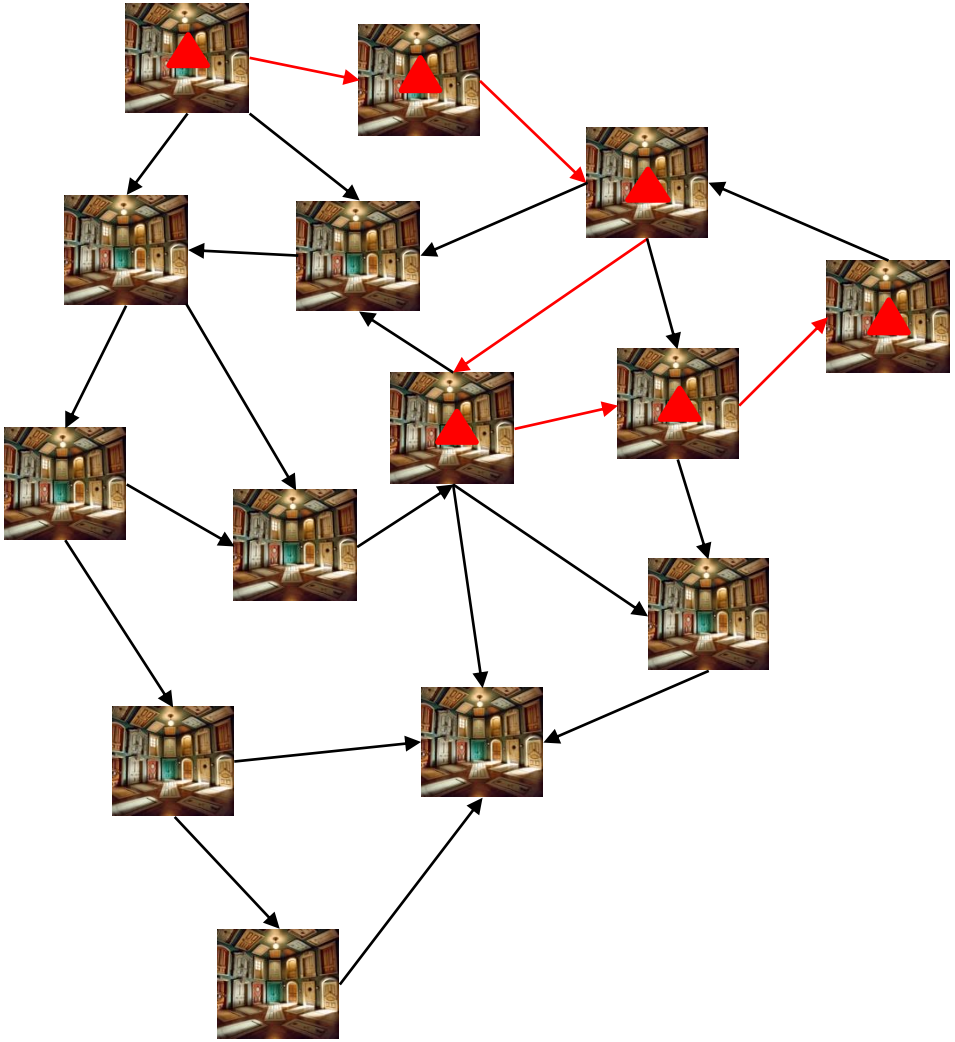




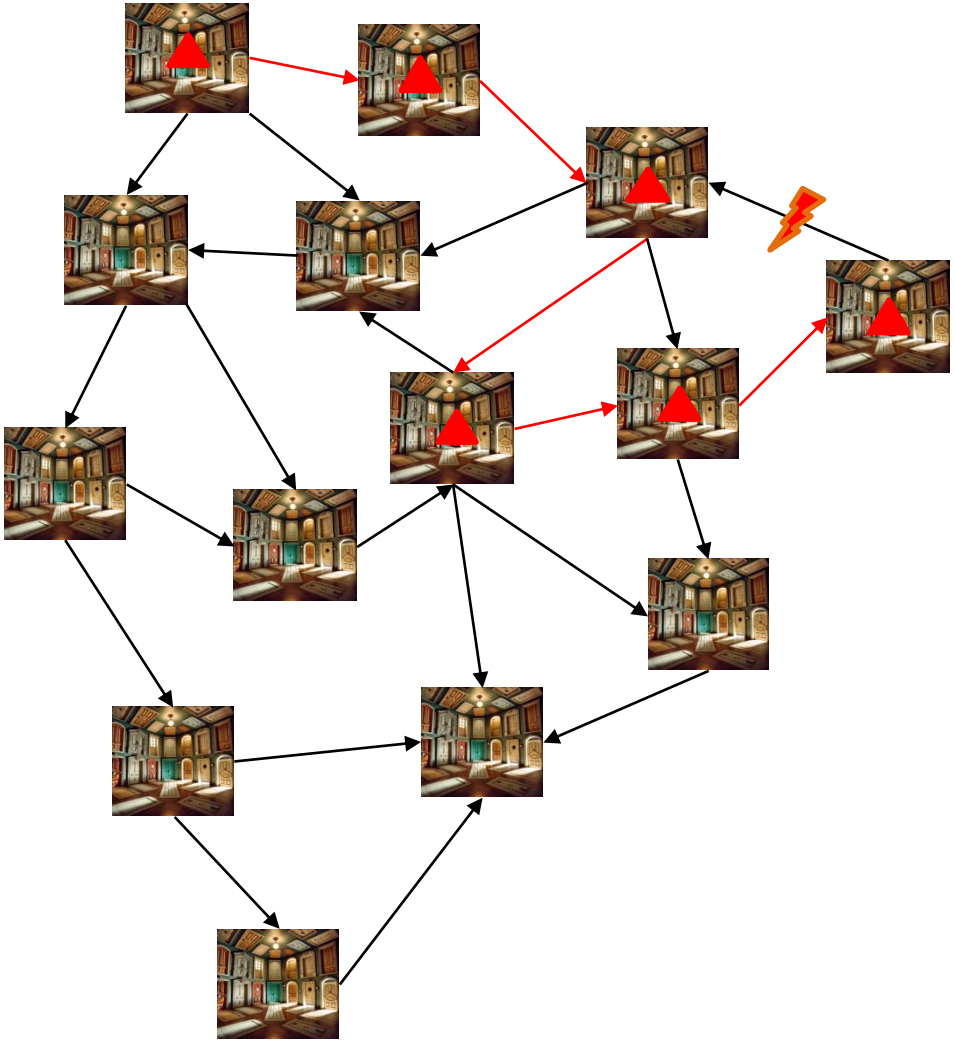




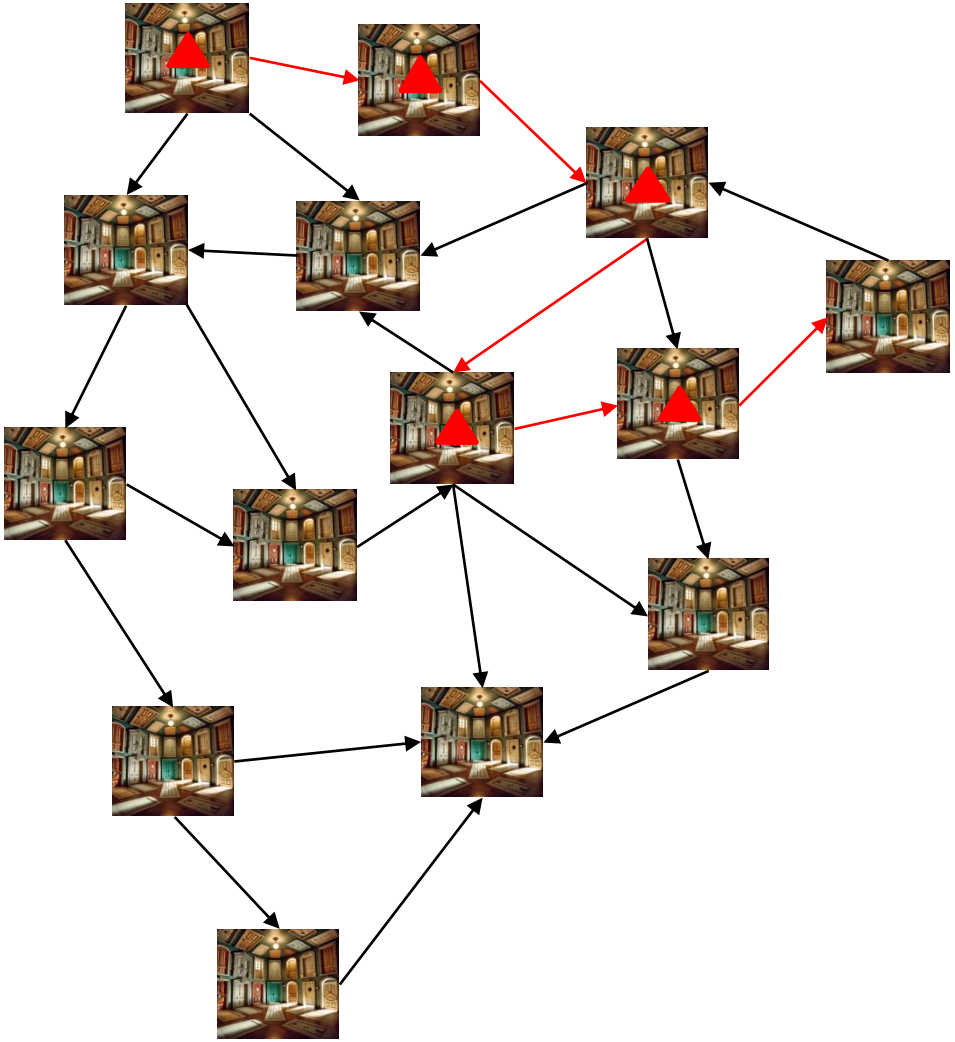




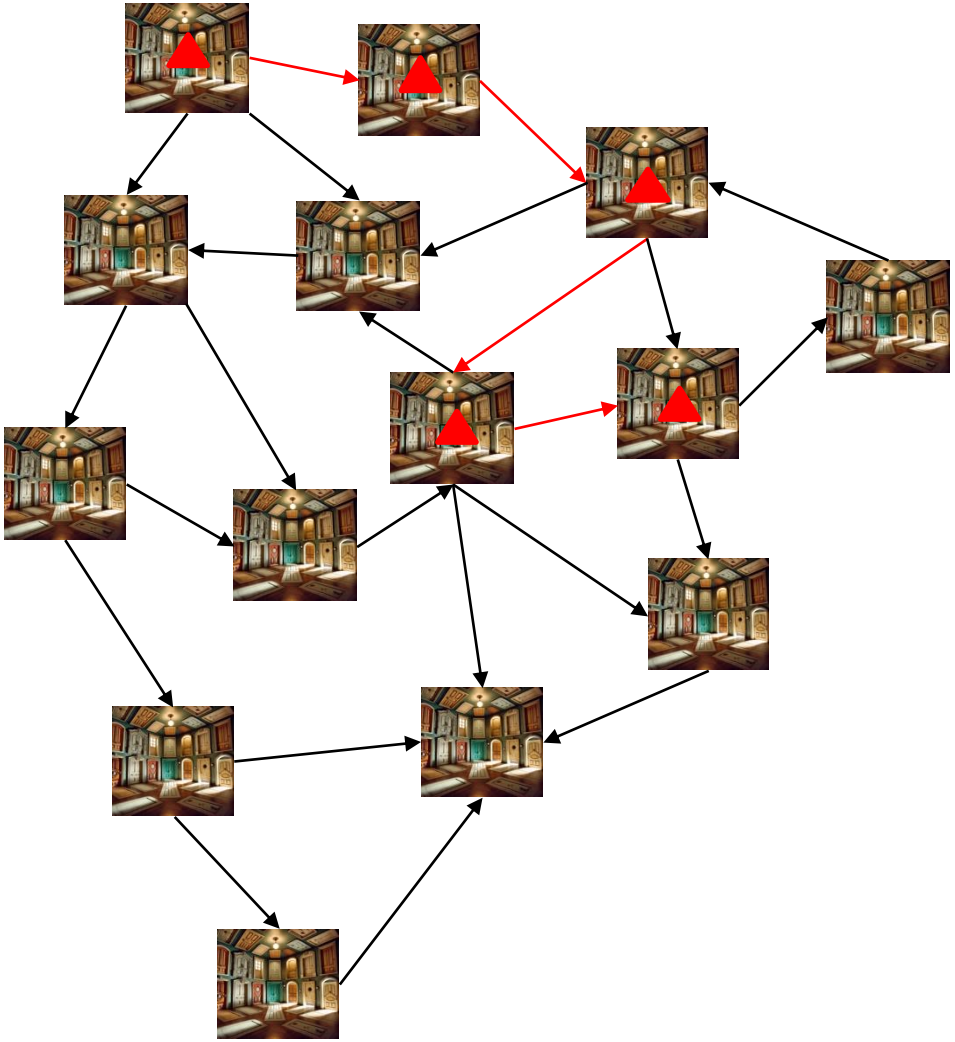
Why visited here?



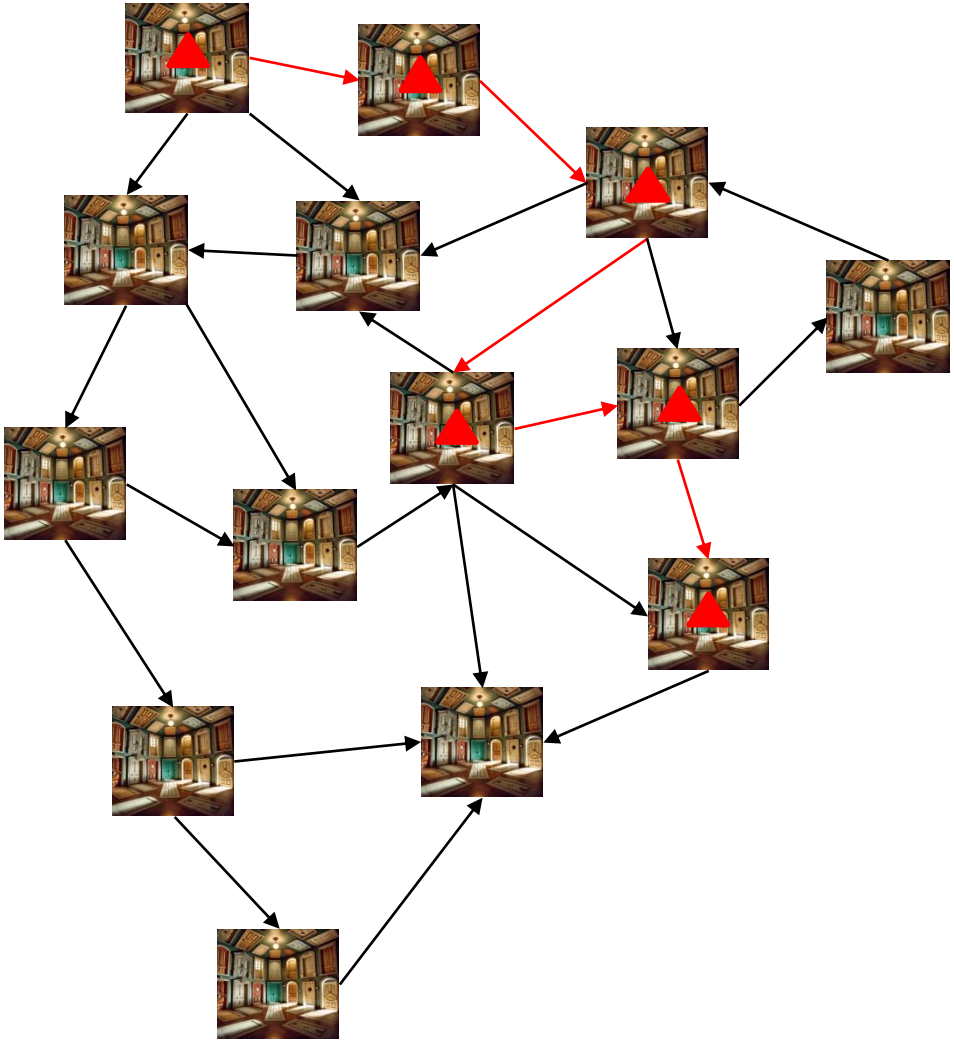
Why visited here?

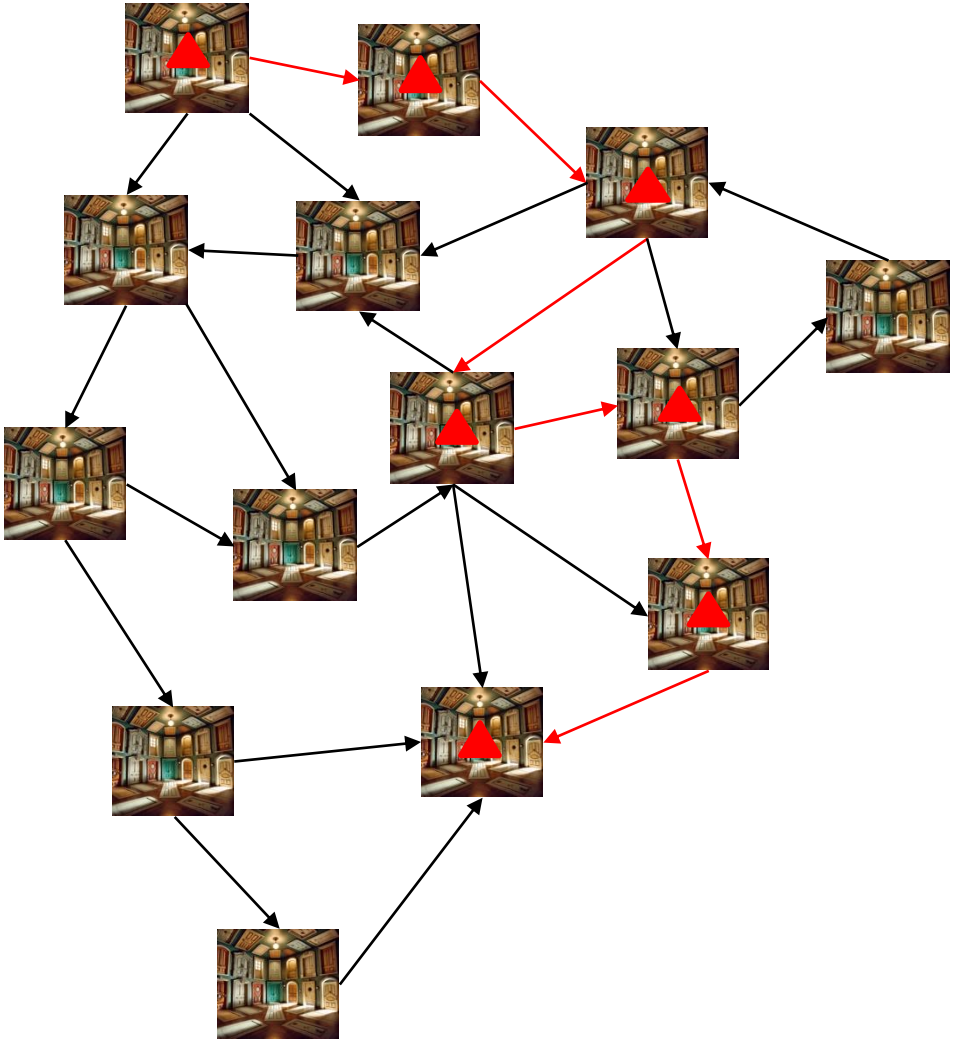


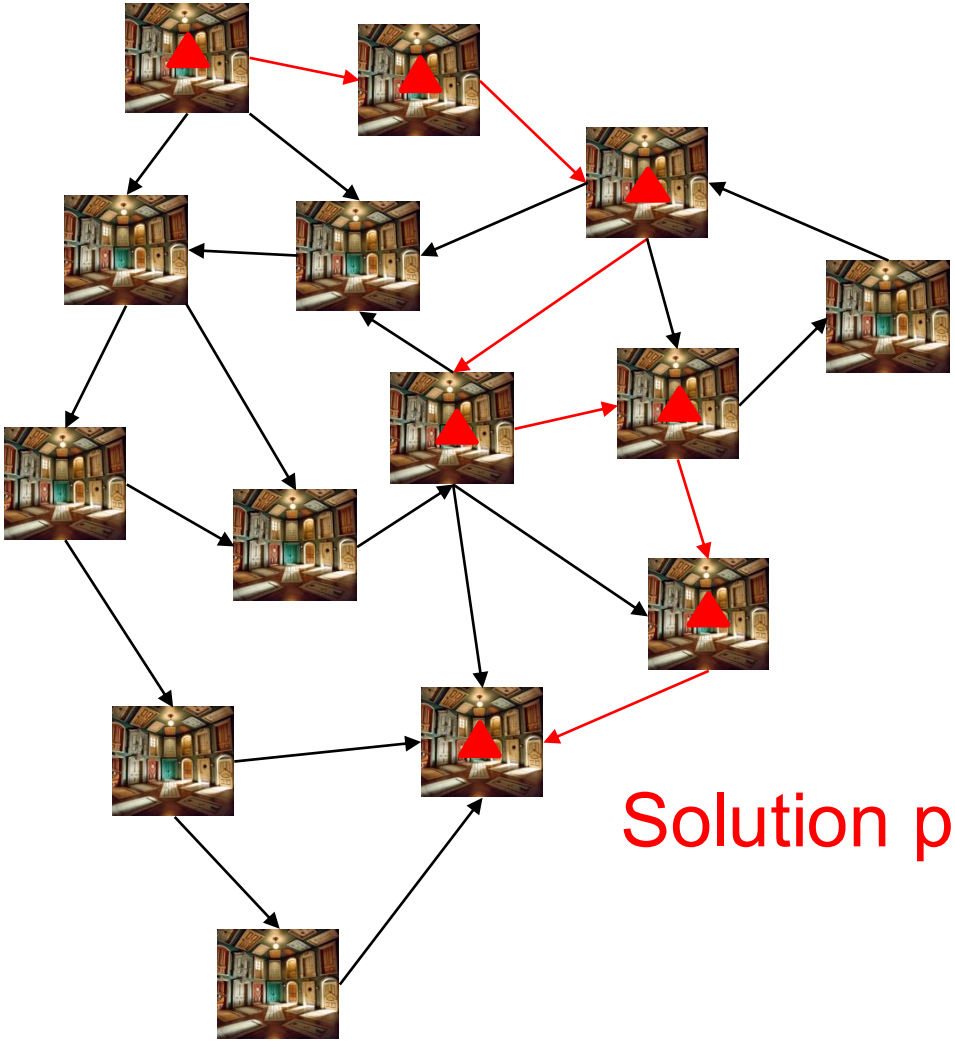
Why visited here?



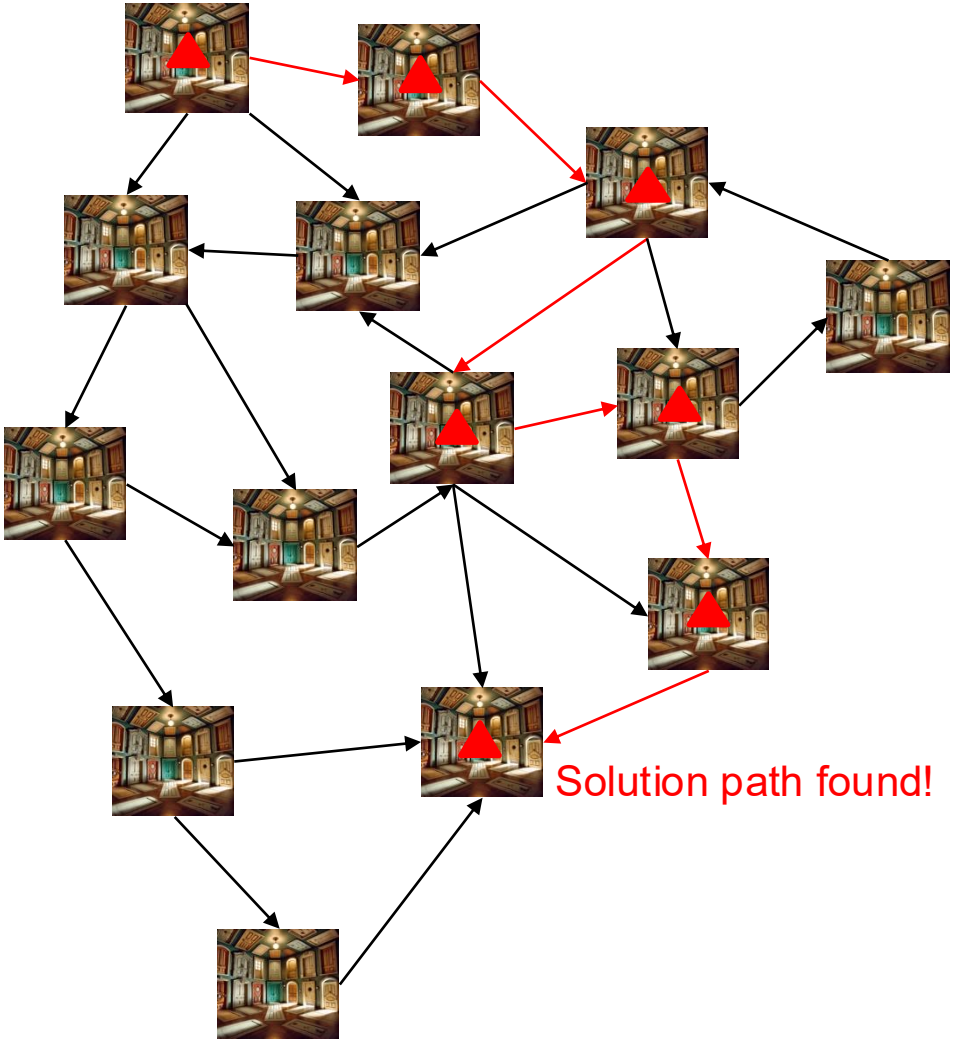
Can we revisit this room
on this path?



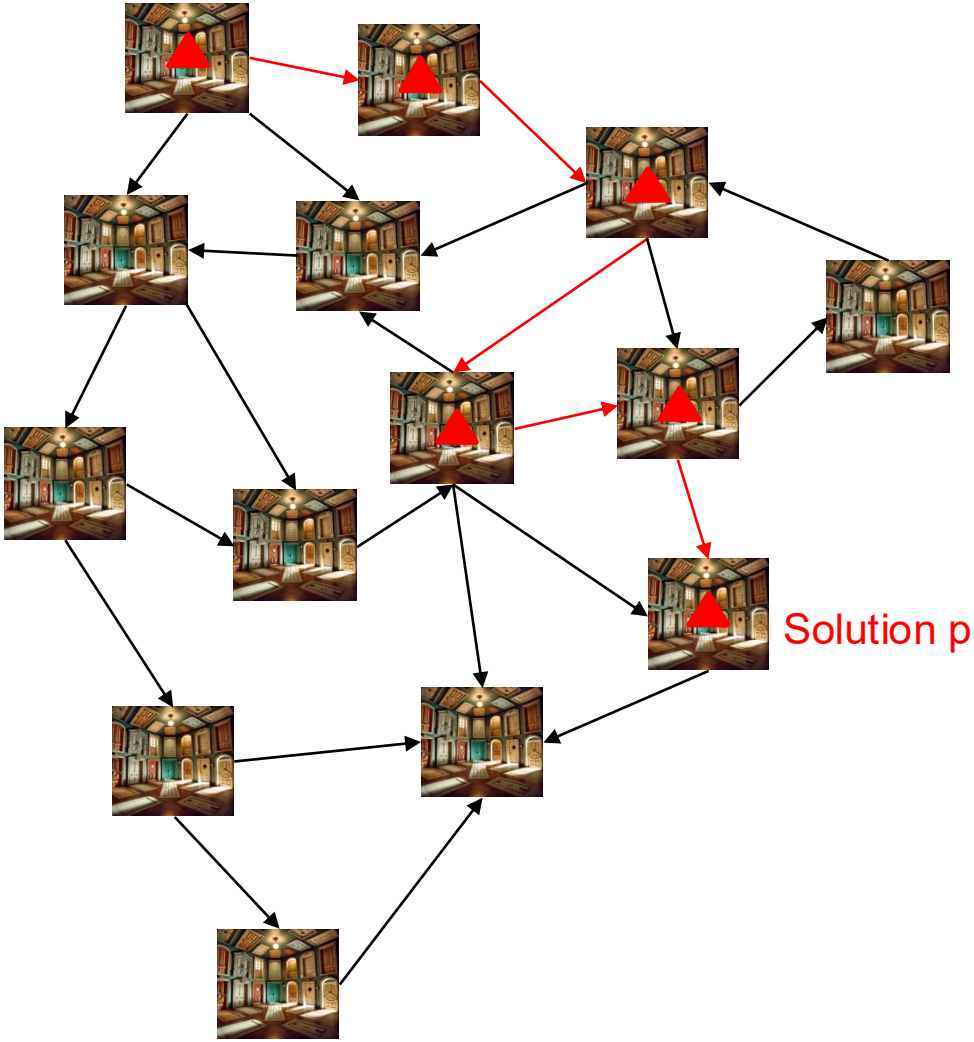




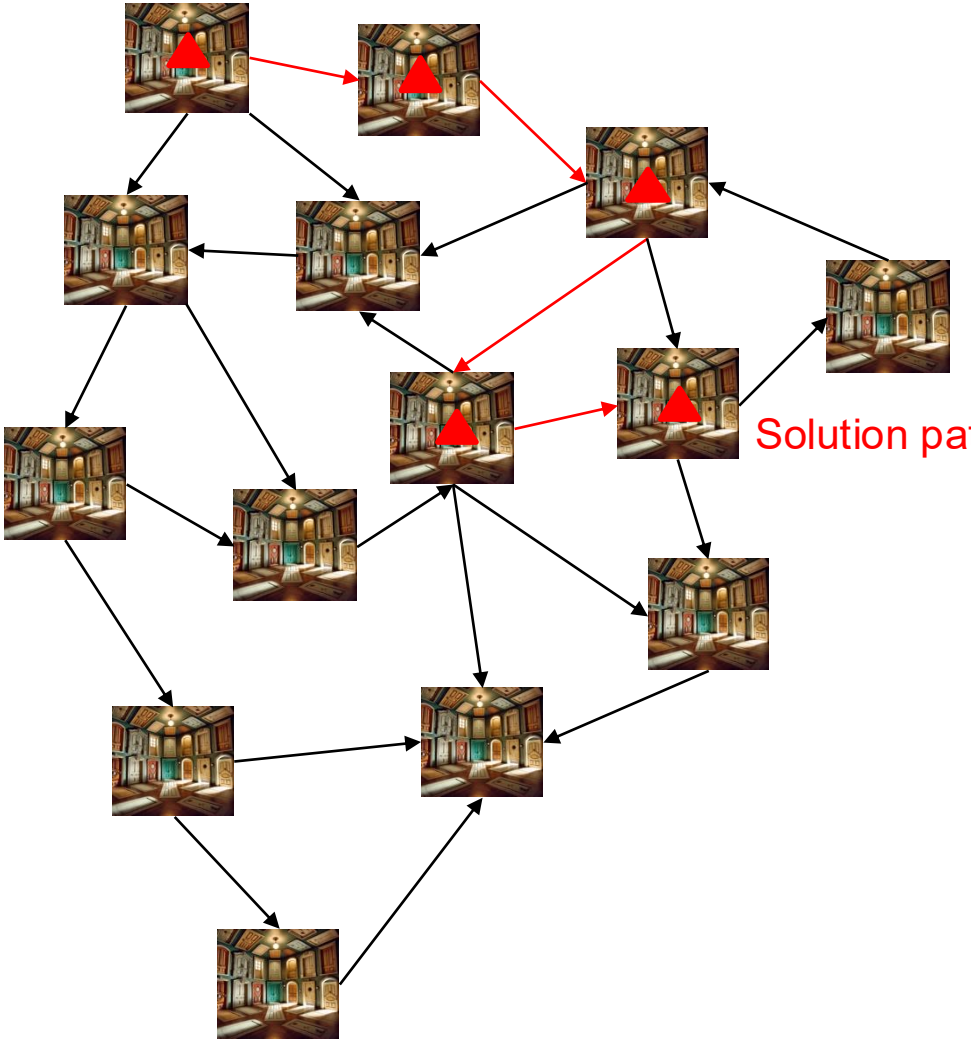
Solution path found!



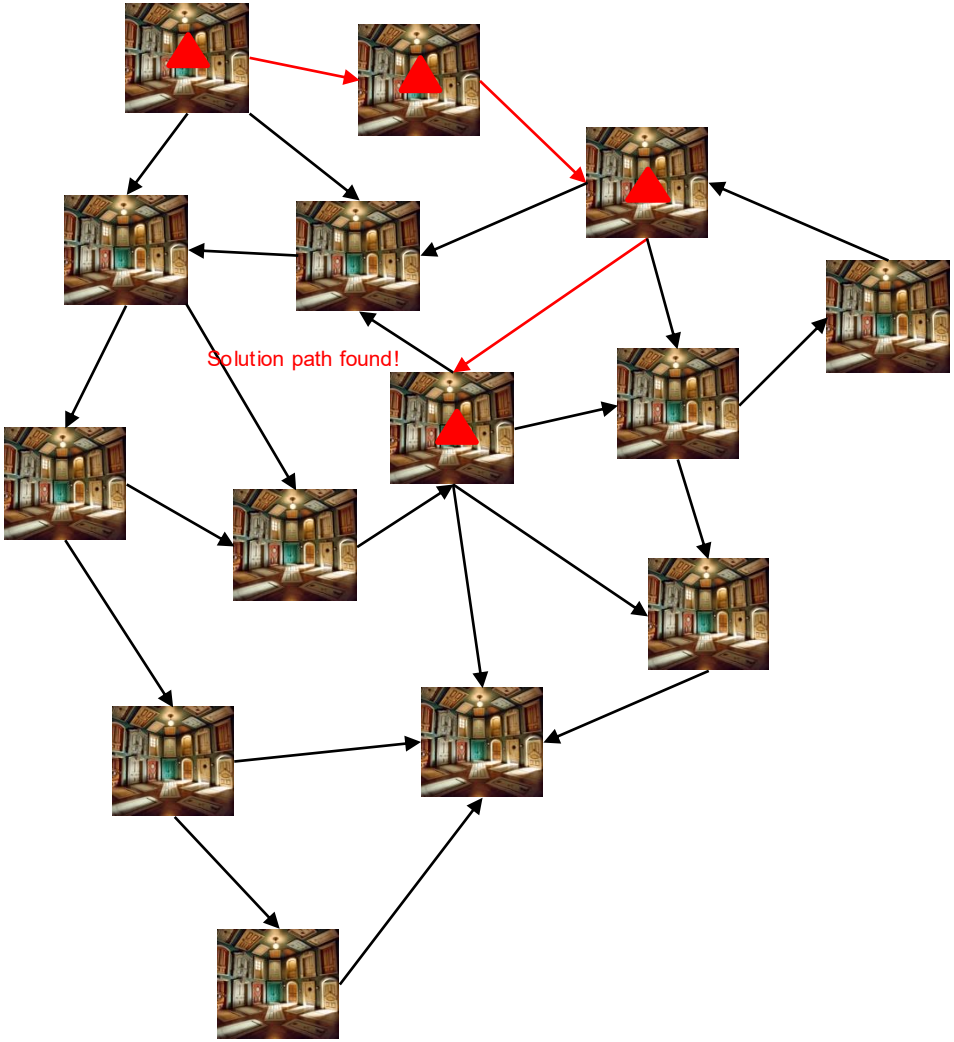
What do we do next?

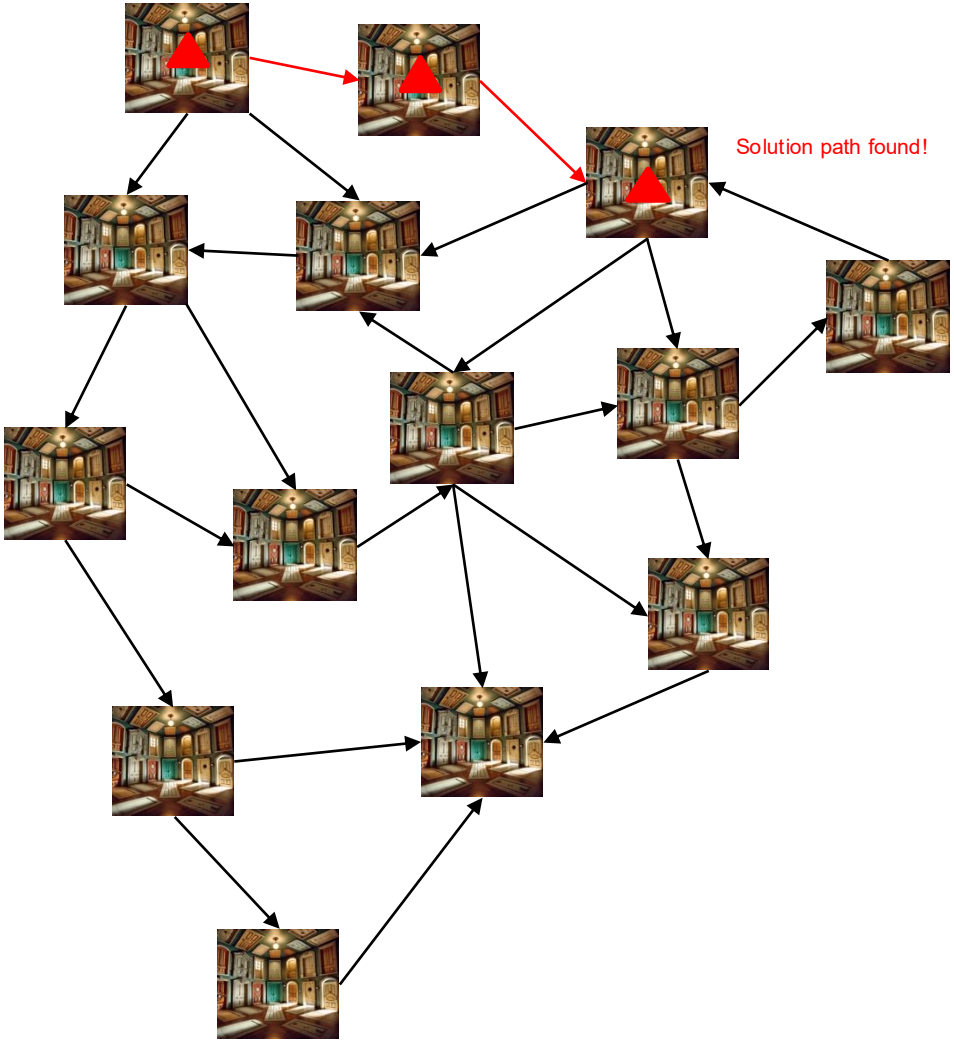


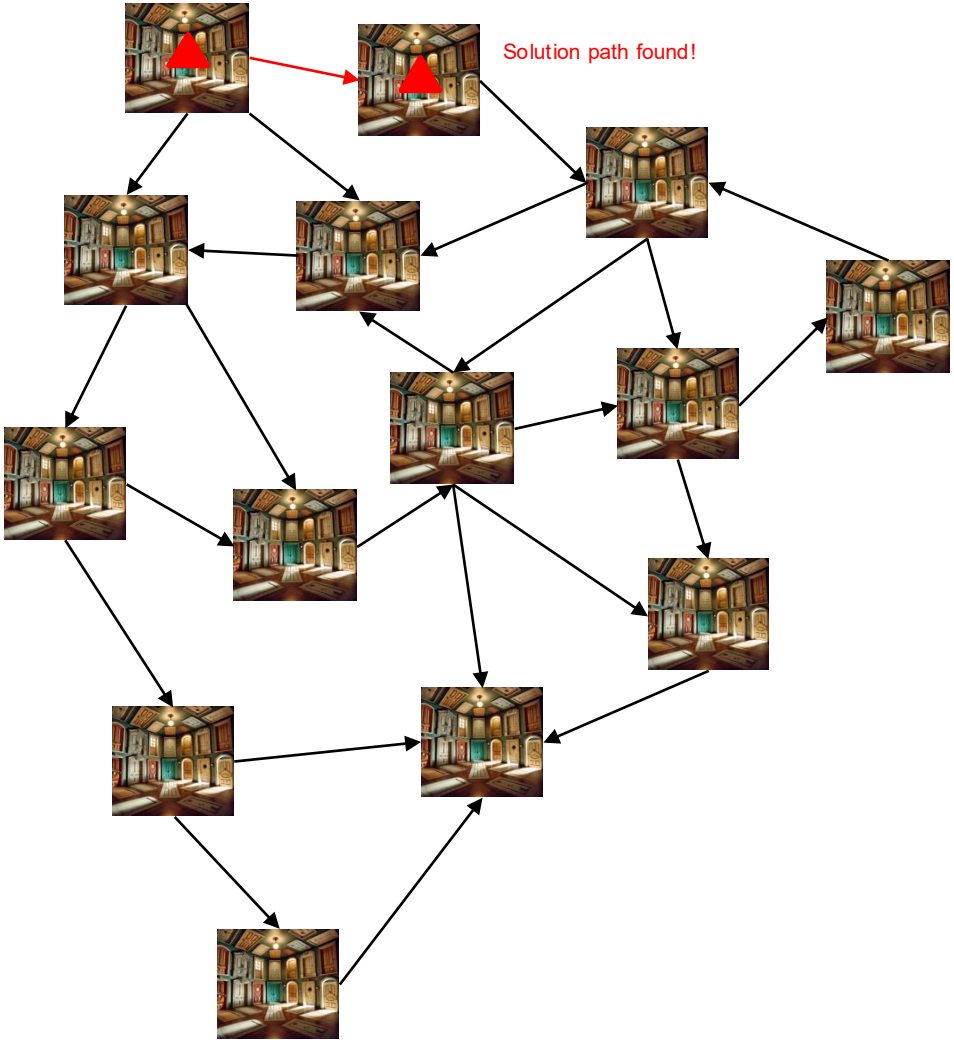
Solution path found!

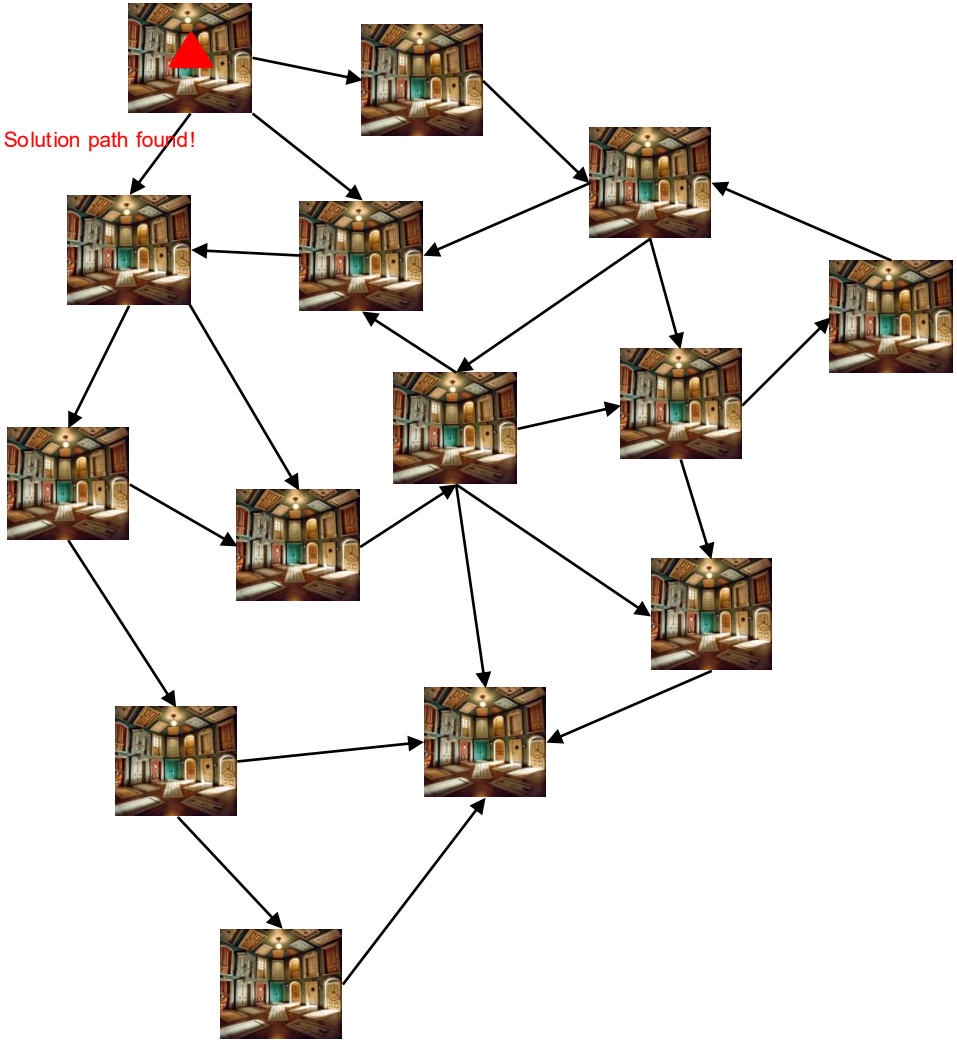


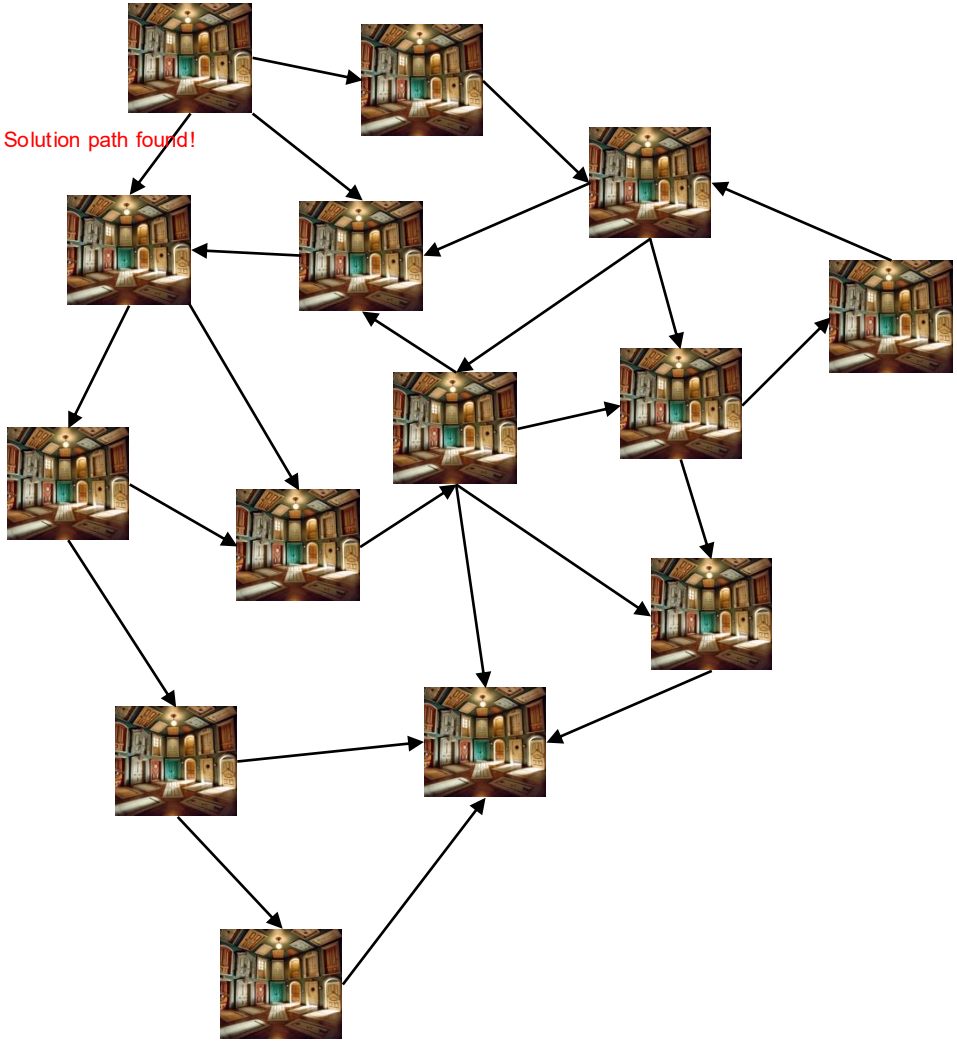
Solution path found!


Solution path found!


Solution path found!


Solution path found!


Solution path found!


Solution path found!

```java
public class Room {

    boolean visited = false; // For cycles: On current path? Reset on return.

    private int age;          // Age of the room, >= 0
    public Room[] doorsTo;    // Doors to other rooms, never null

    public Room(int age, Room... doorsTo) { // PRE: age >= 0; doorsTo and elements != null
        this.age = age;
        this.doorsTo = doorsTo;
    }

    public boolean isExit() { // Exit if there are no doors
        return doorsTo.length == 0;
    }

    public int getAge() { // Return the room's age
        return age;
    }
}
```

Initialisiert das `visited` Attribut für jedes Objekt mit `false`.

```java
public static boolean solve1(Room room, int sum) {
    if (!room.visited) {
        room.visited = true; // No cycle, mark room as visited

        sum = sum + room.getAge();
        if (sum % 3 == 0) {
            room.visited = false; // Reset so other search branches can visit this room again
            return false;
        }
        if (room.isExit()) {
            room.visited = false; // Restore original state
            return true;
        }

        for (int i = 0; i < room.doorsTo.length; i++) {
            if (solve1(room.doorsTo[i], sum)) {
                room.visited = false; // Restore original state
                return true;
            }
        }
        room.visited = false; // Reset so other search branches can visit this room again
    }
    return false; // Room already on current path (cycle) or no path leads to an exit
}
```

Enums

Context: A Possible Game

- Players have
  - An arbitrary name
  - A role: honest or trickster
  - An energy level that rises or falls during the game
    - The change also depends on the role
- Game stores all players in Player[] players
- How should we represent player roles?

```
1 public class Player {
2   private ??? name;
3   private ??? role;
4   private int energy;
5   …
  }
```

First Version: Player and Game

```java
public class Player {
  private String name;
  private String role;
  private int energy;
  …
  public void energize(int value) {
    energy += value;
  }
}
```

Roles as strings
a good idea?

```java
public class Game {
    private Player[] players;
    private Random random;
    …
    public void energizeAll() {
        for (int i = 0; i < players.length; i++) {
            Player player = players[i];
            if (player.role.equals("HONEST")) {
                player.energize(1);
            } else if (player.role.equals("TRICKSTER")) {
                player.energize(random.nextInt(-4, 5));
            }
        }
    }
}
```

Better: With Enums

```
public enum Role {
  HONEST,
  TRICKSTER,
  SOCERER,
}
```

```
public class Player {
  private String name;
  private Role role;
  private int energy;
  …
}
```

Advantages?

```
1  public class Game {
2      private Player[] players;
3      private Random random;
4      …
5      public void energizeAll() {
6          for (int i = 0; i < players.length; i++) {
7              Player player = players[i];
8              if (player.role == Role.HONEST) {
9                  player.energize(1);
10             } else if (player.role == Role.TRICKSTER) {
11                 player.energize(random.nextInt(-4, 5));
12             }
           }
       }
   }
```

Exercise Preview

Exercise 1: Loop Invariant

Note: This task is completed as a Moodle quiz.

Exercise 2: Linked List

A linked list consists of several objects that have references to each other. For this task, each list consists of a "list object" of the class `LinkedIntList`, which represents the entire list, and several "node objects" of the class `IntNode`, one for each value in the list. The list is called "linked" because each node object contains a field with a reference to the next node in the list. Finally, the `LinkedIntList` object contains a reference to the first and the last node and also has a field for the length of the list.

Figure 1 shows a list containing the values 1, 3, 3, 7. Note that the `next` field of the last node in the list points to no object, i.e. contains the value `null`. In addition, an empty list is represented such that both fields `first` and `last` contain the value `null` (and `size` is equal to `0`).

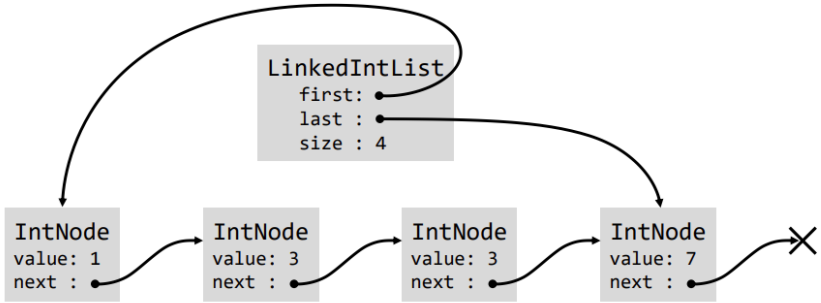


Figure 1: Linked list with values 1, 3, 3, 7.

Exercise 2: Linked List

1. Implement the class `LinkedIntList` (in "LinkedIntList.java"), which together forms a linked list of `int`s. Add the required fields to the respective classes and then implement the following methods in `LinkedIntList` to make the class more user-friendly:

Name	Parameter	Return type	Description
`addLast`	`int value`	`void`	Inserts a node with value `value` at the end of the list.
`addFirst`	`int value`	`void`	Inserts a node with value `value` at the beginning of the list.
`removeFirst`		`int`	Removes the first node and returns its value.
`removeLast`		`int`	Removes the last node and returns its value.
`clear`		`void`	Removes all nodes in the list.
`isEmpty`		`boolean`	Returns whether the list is empty.
`get`	`int index`	`int`	Returns the value of the node at position `index`.
`set`	`int index, int value`	`void`	Replaces the value of the node at position `index` with `value`.
`getSize`		`int`	Returns how many nodes the list contains.

Some of these methods may not be called under certain conditions. For example, `removeFirst()` may not be called when the list is empty, and `get()` may not be called when the given index is greater than or equal to the current length of the list. In such situations, your program should terminate with an error message. Use the following code fragment for this:

```
if(condition) {
    Errors.error(message);
}
```

Replace *`condition`* with the condition under which the program should be terminated, and *`message`* with a helpful error message. The `Errors` class is already located in your project, but you do not need to understand it at the moment.

You will find some tests for the linked list in "LinkedIntListTest.java".

Exercise 3: Executable Graph

In this task, we use directed acyclic graphs to represent programs. The program state is always given by a tuple $(sum, counter)$, where *sum* and *counter* are integers. Program states are modeled by `ProgramState` objects, where `ProgramState.getSum()` (respectively `ProgramState.getCounter()`) corresponds to the first element (respectively the second element) of the tuple.

An execution of the program manipulates the program state. The result of a program is given by the reached program state after all operations in the program have been executed. Programs can be nondeterministic: that is, for a single program and the same initial state, there can be several program executions that lead to different results.

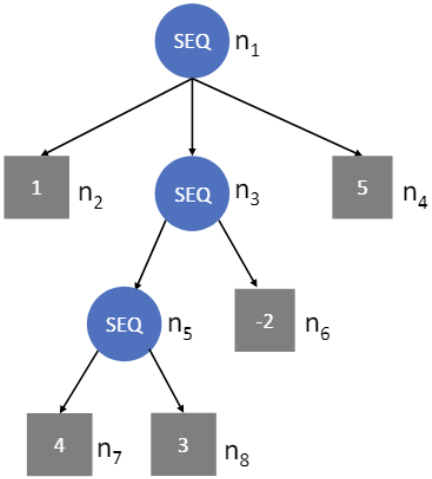
Nodes in graphs are modeled by `Node` objects. `Node.getSubnodes()` returns the child nodes as an array (m is a child node of n exactly when there is an outgoing directed edge from n to m). We distinguish three kinds of nodes, where the method `Node.getType()` returns the node kind as a `String`. To execute a program represented by the node `n`, one must "execute node `n`". We now describe the three node kinds and the execution of each kind of node:

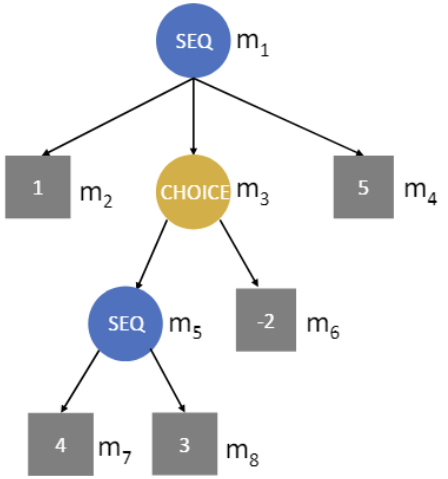
Exercise 3: Executable Graph

1. Addition nodes (`Node.getType()` is "ADD"): Such nodes have an addition value a given by `Node.getValue()` (an integer), and when this node is executed, the program state is updated from $(sum, counter)$ to $(sum + a, counter + 1)$. The child nodes of such nodes are ignored during execution.
2. Sequence nodes (`Node.getType()` is "SEQ"): When a sequence node `n` is executed, the child nodes of `n` are executed one after another. The order in which the child nodes are executed does not matter, because the reached program state is the same for every order. `Node.getValue()` is irrelevant.
3. Choice nodes (`Node.getType()` is "CHOICE"): When a choice node `n` is executed, an arbitrary child node of `n` is selected and executed. `Node.getValue()` is irrelevant. These nodes lead to nondeterminism.

You may assume that sequence and choice nodes always have at least one child node, and that there is always at most one path between two nodes. The following figure shows two example graphs, where nodes labeled "SEQ" (respectively "CHOICE") correspond to sequence nodes (respectively choice nodes), and the numbers in addition nodes correspond to the addition values.

Exercise 3: Executable Graph





Exercise 4: Energy Game

In this task, you practice working with enums. For this, you have a folder `EnergieSpiel` with three classes `GameApp`, `Game`, and `Player`, as well as an enum `Role`. These are already implemented so that everything works. However, the class `Player` has a field `role` of type `String`. Java therefore allows any string to be stored in this field. The game actually has exactly three possibilities: `HONEST`, `TRICKSTER`, or `SORCERER`. The enum `Role` with these three options already exists. Change the type of the field to `Role` and adapt the code in all three classes so that the role logic uses the type `Role` instead of `String` everywhere.

Exercise 5: Comparing Methods

Note: This task is completed as a Moodle quiz.

Exercise 6: Bonus

Attention: This task gives bonus points (see "Performance assessment" in www.vvz.ethz.ch). The task must be solved independently and alone. Submission takes place as usual by pushing to your Git repository on the ETH server. The last push before the submission deadline is binding. Even if you commit before the deadline but push after the deadline, this counts as a late submission. Please also read the general rules.

Exercise Review

Exercise 1: Close Neighbors

Write a program that, for a sorted sequence X of `int` values $(x_1, x_2, ..., x_n)$ and an `int` value `key`, returns the three distinct elements x_a, x_b, and x_c from X that are closest to the value `key`. For x_a, x_b, and x_c, it must hold that $|\texttt{key} - x_a| \leq |\texttt{key} - x_b| \leq |\texttt{key} - x_c| \leq |\texttt{key} - x_i|$ for all $i \neq a, b, c$ and that $x_a \neq x_b \neq x_c \neq x_a$. If the three values are not uniquely determined, then any solution satisfying the above condition is allowed.

Examples:
The closest neighbors for `key == 5` in $(1, 4, 5, 7, 9, 10)$ are $5, 4, 7$.
The closest neighbors for `key == 5` in $(1, 4, 5, 6, 9, 10)$ are $5, 4, 6$ or $5, 6, 4$.
The closest neighbors for `key == 10` in $(1, 4, 5, 6, 9, 10)$ are $10, 9, 6$.

Implement the computation in the method `int[] neighbor(int[] sequence, int key)`, which is located in the class `Neighbor`. The method declaration is already given. You can assume that the argument `sequence` is not `null`, is sorted, contains only distinct elements, and contains at least three elements. Remember that the value `key` does not necessarily occur in the sequence X. You may not change the input array `sequence`.

In the `main` method of the class `Neighbor`, you will find the examples above as small tests that make example calls to the `neighbor` method and that you can use as a basis for further tests. In the file `NeighborTest.java`, we additionally provide the same tests as JUnit tests. You may also change these as you like. You are *not* expected to use the JUnit test for this task.

Exercise 2: Loop Invariant

1. The precondition and postcondition are given for the following program.

```
public int compute(int n) {
    // Precondition:  n >= 0
    int x;
    int res;

     x = 0;
    res = x;

    // Loop Invariant:
    while (x <= n) {
        res = res + x;
        x = x + 1;
    }
    // Postcondition:  res == ((n + 1) * n) / 2
    return res;
}
```

Write the loop invariant in the file “LoopInvariante.txt”.

Exercise 2: Loop Invariant

2. The precondition and postcondition are given for the following program.

```
public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = 0;
    res = b;

    // Loop Invariant:
    while (x < a) {
        res = res - 1;
        x = x + 1;
    }
    // Postcondition:  res == b - a
    return res;
}
```

Write the loop invariant in the file "LoopInvariante.txt".

Exercise 3: Bills

In this task, you should implement part of the system that creates the bills for the local electricity provider.

Complete the `process` method in the class `Bills`. The method has two arguments: a `Scanner` from which you should read the contents of the input file, and a `PrintStream` to which you write the information described below.

Your program only needs to support correctly formatted input files. You will find an example of such a file in the project under the name "Data.txt". You may ignore exceptions related to input and output.

A valid input file contains lines that describe either a tariff or the electricity consumption of a customer. The consumption of a customer is never greater than 100000 kilowatt-hours.

A tariff description has the following format:

$$\texttt{Tarif}\ n\ l_1\ p_1 \ldots l_n\ p_n$$

Exercise 4: Game (Bonus)

Task 6: Game (Bonus!)

Attention: This task gives bonus points (see "Performance assessment" in `www.vvz.ethz.ch`). The task must be solved independently and alone. Submission takes place as usual by pushing to your Git repository on the ETH server. The last push before the submission deadline is binding. Even if you commit before the deadline but push after the deadline, this counts as a late submission. Please also read the general rules.